

RIGA TECHNICAL UNIVERSITY

Faculty of Computer Science, Information Technology and Energy

Report on the second practical assignment

Study course "Fundamentals of artificial intelligence"

Team number: Team 22

Students:

- Ngoc Bao Tram, Tran, 231ADB294
- Kamalesh Ashokkumar Kowsalya, 221ADB216
- Bakhodir Rasulov, 231ADB155

Teaching staff: Alla Anohina-Naumeca

Project link:

[Link of the dataset](#)

[Link to the code \(Preprocessing\)](#)

[Link to the Orange Tool's project](#)

2024/2025 academic year

Statement of the use of AI tools

AI Model Contributions Analysis

1. ChatGPT

- **Model Info:** ChatGPT-4o-latest (May 13, 2024)
- **Source:** [OpenAI – Hello GPT-4o](#)
- **Contributions:**
 1. **Scatter Plot Analysis** (Page 34)
 2. **Histograms, Boxplots, and Statistical Analysis** (Pages 21, 22, 23, 24)
 3. **Hyperparameters for Hierarchical Clustering & K-means + Part 3 Algorithm** (Pages 25, 26, 32, 38)
 4. **Grammar Correction for Algorithm Descriptions** (Pages 36, 37)
 5. **Testing Dataset Evaluation** (Pages 56, 57)
 6. **Evaluating the boxplots to analyse the outliers** (Page 11)
 7. **Representation of features (attributes) of the dataset together with their roles in the orange tool – Analyze which are suitable for the selected features and the reasons** (Page 7)

2. DeepSeek

- **Model Info:** DeepSeek-V3-latest (May 2024)
- **Source:** [DeepSeek Official](#)
- **Contributions:**
 1. **Parameter (Trees) Experiment Analysis** (Pages 45, 46, 47)
 2. **Algorithm Suitability (Random Forest) for Dataset** (Pages 34, 35)
 3. **Logistic Regression Experiment Conclusions** (Page 53)
 4. **Description of the dataset problem domain + Information about the method or procedure for collecting the dataset** (Pages 4, 6)

3. Perplexity AI

- **Model Info:** GPT-4 variant (June 2024)
- **Source:** [Perplexity AI](#)
- **Contributions:**
 1. **Histograms, Boxplots, and Statistical Analysis** (Pages 21, 22, 23, 24)

Orange tool workflow

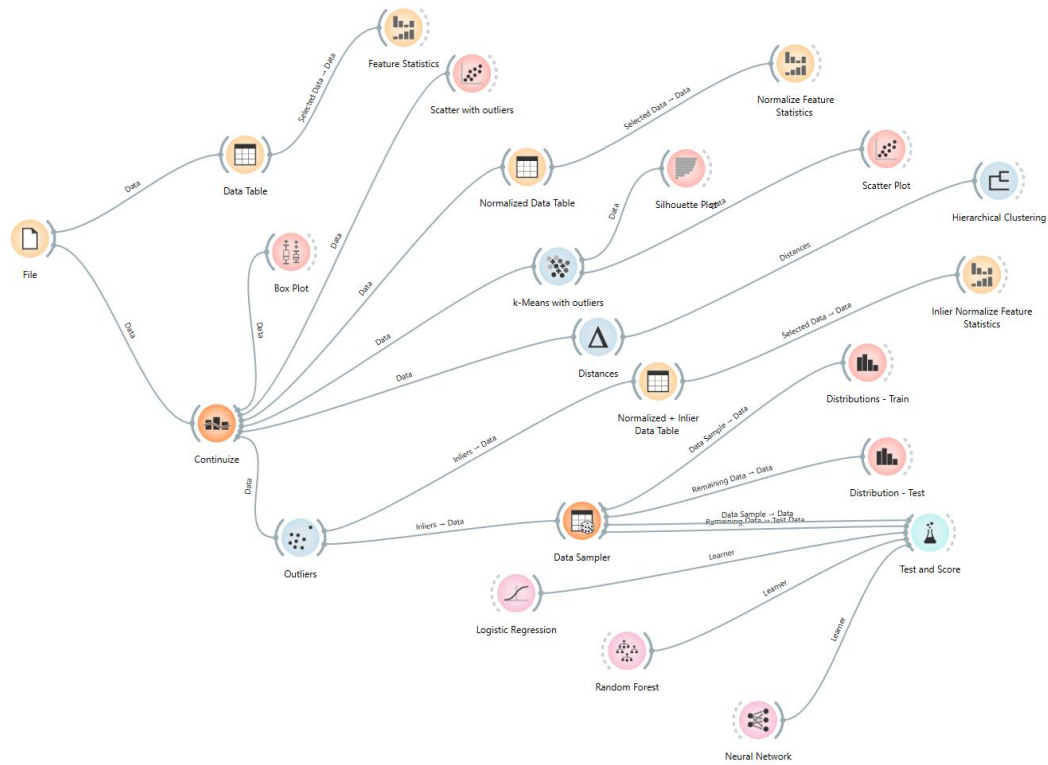


Fig.0. Orange Tool Workflow

Part I

1. Description of the dataset

Description: The dataset used in this project is the **Glass Identification** dataset, which is available on the UCI Machine Learning Repository.

1.1. Dataset title: Glass Identification

1.2. Dataset source: <https://archive.ics.uci.edu/dataset/42/glass+identification>

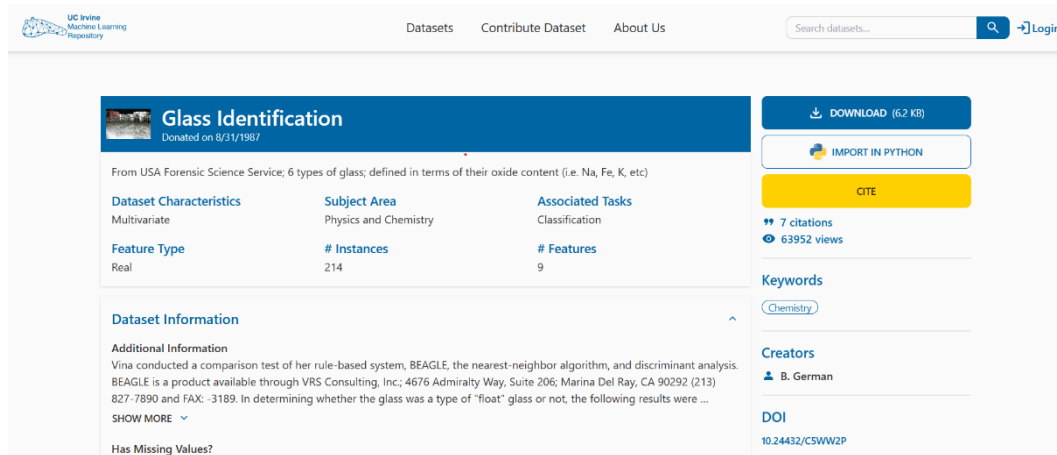


Fig.1 UC Irvine Homepage

1.3. Creator and/or owner of the dataset:

The Glass Identification dataset was created by **B. German**, as noted on the **UCI Machine Learning Repository**. According to the accompanying glass.names file, the data originated from the Central Research Establishment of the Home Office Forensic Science Service, located in Aldermaston, Reading, Berkshire RG7 4PN.

1.4. Description of the dataset problem domain:

This dataset is commonly used in machine learning and pattern recognition for multi-class classification. The aim is to **identify the type of glass** (e.g., window glass, containers, tableware) based on its **chemical properties**. It has applications in forensic science, especially in criminal investigations where glass fragments are found at crime scenes - assisting in criminal investigations. The dataset contains **9 numerical attributes**. The attributes are:

- Refractive Index (RI)
- Sodium (Na)
- Magnesium (Mg)
- Aluminium (Al)
- Silicon (Si)
- Potassium (K)
- Calcium (Ca)
- Barium (Ba)
- Iron (Fe)

Variables Table					
Variable Name	Role	Type	Description	Units	Missing Values
Id_number	ID	Integer			no
RI	Feature	Continuous	refractive index		no
Na	Feature	Continuous	Sodium	weight percent in corresponding oxide	no
Mg	Feature	Continuous	Magnesium	weight percent in corresponding oxide	no
Al	Feature	Continuous	Aluminum	weight percent in corresponding oxide	no
Si	Feature	Continuous	Silicon	weight percent in corresponding oxide	no
K	Feature	Continuous	Potassium	weight percent in corresponding oxide	no
Ca	Feature	Continuous	Calcium	weight percent in corresponding oxide	no
Ba	Feature	Continuous	Barium	weight percent in corresponding oxide	no
Fe	Feature	Continuous	Iron	weight percent in corresponding oxide	no
Rows per page 10 0 to 10 of 11 < >					

Variables Table					
Variable Name	Role	Type	Description	Units	Missing Values
Type_of_glass	Target	Categorical			no
Rows per page 10 10 to 11 of 11 < >					

Fig.1.1 Dataset Problem Domain

1.5. Dataset licensing conditions:

This dataset is licensed under a [Creative Commons Attribution 4.0 International](https://creativecommons.org/licenses/by/4.0/) (CC BY 4.0) license. This allows for the sharing and adaptation of the datasets for any purpose, provided that the appropriate credit is given.

License

This dataset is licensed under a [Creative Commons Attribution 4.0 International](#) (CC BY 4.0) license.

This allows for the sharing and adaptation of the datasets for any purpose, provided that the appropriate credit is given.

Fig.1.2 License

1.6. Information about the method or procedure for collecting the dataset:

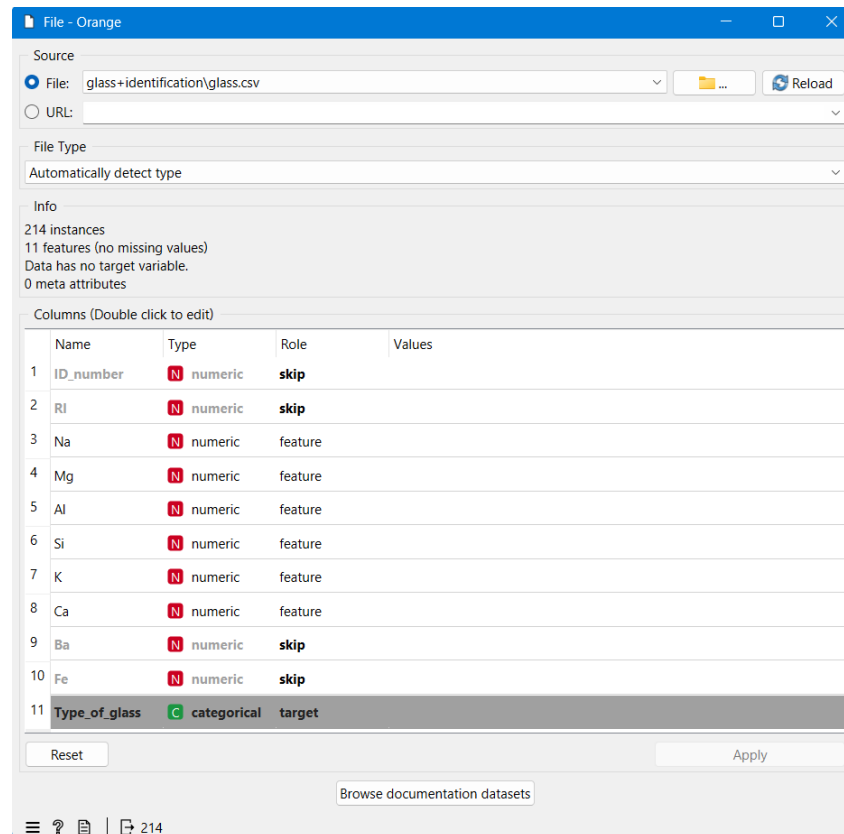
The dataset was compiled by measuring the chemical composition and refractive index of various glass samples. These samples were then classified according to their intended use - such as building windows, containers, or tableware. The chemical properties were determined using spectrometry and other laboratory techniques. Classification labels were assigned manually by experts, based on each sample's origin and manufacturing process.

The dataset we retrieved from the UCI repository was originally in a .dat file format, which is not convenient for direct analysis. Therefore, we converted it into a more usable .csv format to facilitate processing.

2. Description of the dataset content

2.1. Number of data objects in the dataset: 214

2.2. Representation of features (attributes) of the dataset together with their roles in the orange tool:



	Name	Type	Role	Values
1	ID_number	numeric	skip	
2	RI	numeric	skip	
3	Na	numeric	feature	
4	Mg	numeric	feature	
5	Al	numeric	feature	
6	Si	numeric	feature	
7	K	numeric	feature	
8	Ca	numeric	feature	
9	Ba	numeric	skip	
10	Fe	numeric	skip	
11	Type_of_glass	categorical	target	

Fig.1.3 Selected features

We choose these features:

Si – Has a nearly normal distribution, few outliers, good class separation.

Al – Stable distribution, mean $\approx$ median $\rightarrow$ not skewed, suitable for modeling.

Na – Evenly distributed, not noisy.

Ca – Some outliers, but overall, a good distribution.

Mg – Many zero values (left-skewed), but shows class separation in some plots.

K – Skewed and tightly packed at low values $\rightarrow$ should be normalized or log-transformed if included.

We don't choose these features:

Fe – Has many zeros and outliers.

Ba – Too many zeros, highest dispersion, very noisy $\rightarrow$ likely to harm model performance unless carefully handled.

RI (Refractive Index) – Although it shows variation across classes (an useful feature), to avoid multicollinearity and redundancy, it is advisable to exclude one feature from highly correlated pairs. RI and Ca are highly correlated; we might choose to keep only one of them for modelling.

2.3. Number of classes in the dataset:

7 classes due to UC Irvine Machine Learning Repository. Class Labels will be 1, 2, 3, 4, 5, 6 and 7. The dataset contains samples from 6 classes. Class 4 is part of the overall class label set, but it does not appear in this dataset.

Class Label	Glass Type	Samples (Count)	Notes
1	building_windows_float_processed	70	Most common class.
2	building_windows_non_float_processed	77	Second most common.
3	vehicle_windows_float_processed	17	Fewer samples.
4	vehicle_windows_non_float_processed	0	No samples in this dataset.
5	containers (e.g., bottles)	13	Rare class.
6	tableware (e.g., drinking glasses)	9	Very rare class.
7	headlamps	29	Present in some versions.

2.4. Description of classes:

We classify the glass_type – it is the target of this dataset.

Class Label	Glass Type	Description
1	building_windows_float_processed	Windows made using the float glass process (molten glass floated on tin).
2	building_windows_non_float_processed	Windows made using older methods (e.g., sheet or rolled glass).
3	vehicle_windows_float_processed	Vehicle windows (e.g., car windshields) made via the float process.
4	vehicle_windows_non_float_processed	<i>No samples in this dataset</i>
5	containers	Glass used for bottles, jars, or other containers.
6	tableware	Glass for drinking glasses, dishes, or decorative items.

Class Label	Glass Type	Description
7	headlamps	Vehicle headlamp glass (included in some dataset versions).

2.5. Number of data objects belonging to each class:

Class label – Type_of_glass	Number of data objects
2 - building_windows_non_float_processed	77
1 - building_windows_float_processed	70
7 - headlamps	29
3 - vehicle_windows_float_processed	17
5 - containers	13
6 - tableware	9
4 - vehicle_windows_non_float_processed	0

2.6. Description of features:

Feature title	Explanation of the feature	Value type	Range of values
RI	Refractive index	Continuous	[1.51115, 1.53393]
Na	Sodium (unit measurement: weight percent in corresponding oxide)	Continuous	[10.73, 17.38]
Mg	Magnesium (unit measurement: weight percent in corresponding oxide)	Continuous	[0, 4.49]
Al	Aluminum (unit measurement: weight percent corresponding oxide)	Continuous	[0.29, 3.5]
Si	Silicon (unit measurement: weight percent in corresponding oxide)	Continuous	[69.81, 75.41]
K	Potassium (unit measurement: weight percent in corresponding oxide)	Continuous	[0, 6.21]
Ca	Calcium (unit measurement: weight percent in corresponding oxide)	Continuous	[5.43, 16.19]
Ba	Barium (unit measurement: weight percent in corresponding oxide)	Continuous	[0, 3.15]
Fe	Iron (unit measurement: weight percent in corresponding oxide)	Continuous	[0, 0.51]
Type_of_glass	Type of glass - Target class label (glass category in corresponding oxide)	Categorical	[1, 7]

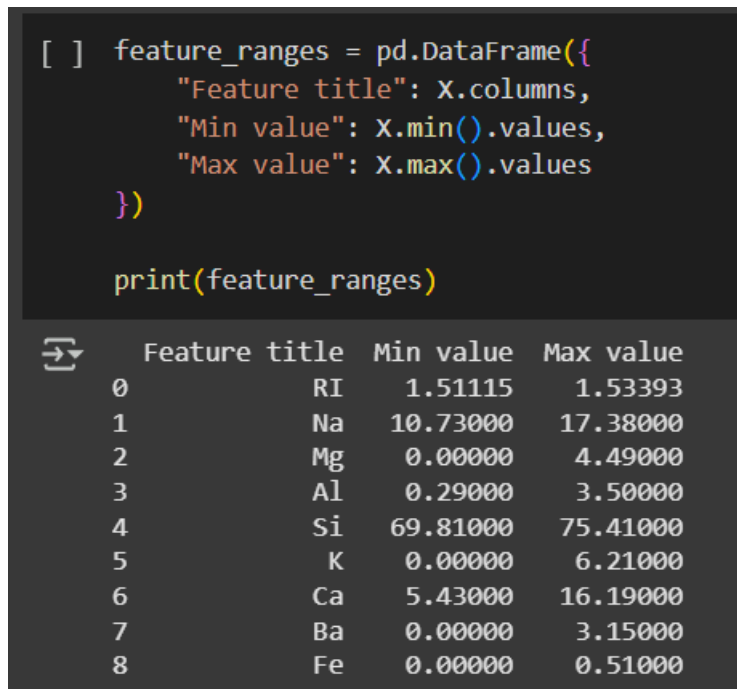


Fig. 1.4 Description of features

2.7. Data file structure:

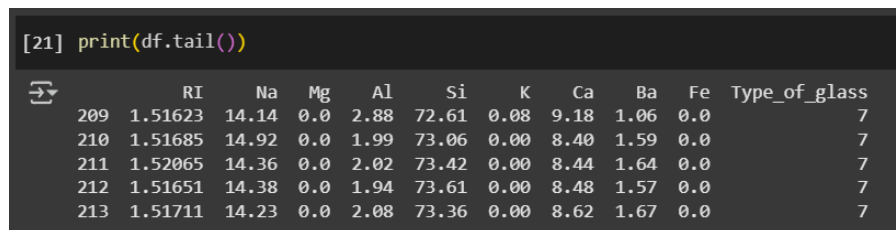


Fig.1.5 Python code

With Orange tool:

	ID_number	RI	Na	Mg	Al	Si	K	Ca	Ba	Fe	Type_of_glass
1	1	1.52101	13.64	4.49	1.10	71.78	0.06	8.75	0.00	0.00	1
2	2	1.51761	13.89	3.60	1.36	72.73	0.48	7.83	0.00	0.00	1
3	3	1.51618	13.53	3.55	1.54	72.99	0.39	7.78	0.00	0.00	1
4	4	1.51766	13.21	3.69	1.29	72.61	0.57	8.22	0.00	0.00	1
5	5	1.51742	13.27	3.62	1.24	73.08	0.55	8.07	0.00	0.00	1
6	6	1.51596	12.79	3.61	1.62	72.97	0.64	8.07	0.00	0.26	1
7	7	1.51743	13.30	3.60	1.14	73.09	0.58	8.17	0.00	0.00	1
8	8	1.51756	13.15	3.61	1.05	73.24	0.57	8.24	0.00	0.00	1
9	9	1.51918	14.04	3.58	1.37	72.08	0.56	8.30	0.00	0.00	1
10	10	1.51755	13.00	3.60	1.36	72.99	0.57	8.40	0.00	0.11	1
11	11	1.51571	12.72	3.46	1.56	73.20	0.67	8.09	0.00	0.24	1
12	12	1.51763	12.80	3.66	1.27	73.01	0.60	8.56	0.00	0.00	1
13	13	1.51589	12.88	3.43	1.40	73.28	0.69	8.05	0.00	0.24	1
14	14	1.51748	12.86	3.56	1.27	73.21	0.54	8.38	0.00	0.17	1
15	15	1.51763	12.61	3.59	1.31	73.29	0.58	8.50	0.00	0.00	1
16	16	1.51761	12.81	3.54	1.23	73.24	0.58	8.39	0.00	0.00	1
17	17	1.51784	12.68	3.67	1.16	73.11	0.61	8.70	0.00	0.00	1
18	18	1.52196	14.36	3.85	0.89	71.36	0.15	9.15	0.00	0.00	1
19	19	1.51911	13.90	3.73	1.18	72.12	0.06	8.89	0.00	0.00	1
20	20	1.51735	13.02	3.54	1.69	72.73	0.54	8.44	0.00	0.07	1
21	21	1.51750	12.82	3.55	1.49	72.75	0.54	8.52	0.00	0.19	1
22	22	1.51966	14.77	3.75	0.29	72.02	0.03	9.00	0.00	0.00	1
23	23	1.51736	12.78	3.62	1.29	72.79	0.59	8.70	0.00	0.00	1
24	24	1.51751	12.81	3.57	1.35	73.02	0.62	8.59	0.00	0.00	1
25	25	1.51720	13.38	3.50	1.15	72.85	0.50	8.43	0.00	0.00	1
26	26	1.51764	12.98	3.54	1.21	73.00	0.65	8.53	0.00	0.00	1
27	27	1.51793	13.21	3.48	1.41	72.64	0.59	8.43	0.00	0.00	1
28	28	1.51721	12.87	3.48	1.33	73.04	0.56	8.43	0.00	0.00	1
29	29	1.51768	12.56	3.52	1.43	73.15	0.57	8.54	0.00	0.00	1
30	30	1.51784	13.08	3.49	1.28	72.86	0.60	8.49	0.00	0.00	1
31	31	1.51768	12.65	3.56	1.30	73.08	0.61	8.69	0.00	0.14	1
32	32	1.51747	12.84	3.50	1.14	73.27	0.56	8.55	0.00	0.00	1
33	33	1.51775	12.85	3.48	1.23	72.97	0.61	8.56	0.09	0.22	1
34	34	1.51753	12.57	3.47	1.38	73.39	0.60	8.55	0.00	0.06	1
35	35	1.51783	12.69	3.54	1.34	72.95	0.57	8.75	0.00	0.00	1

Fig.1.6 Orange tool datasets

	ID_number	RI	Na	Mg	Al	Si	K	Ca	Ba	Fe	Type_of_glass
139	139	1.51674	12.79	3.52	1.54	73.36	0.66	7.90	0.00	0.00	2
140	140	1.51674	12.87	3.56	1.64	73.14	0.65	7.99	0.00	0.00	2
141	141	1.51690	13.33	3.54	1.61	72.54	0.68	8.11	0.00	0.00	2
142	142	1.51851	13.20	3.63	1.07	72.83	0.57	8.41	0.09	0.17	2
143	143	1.51662	12.85	3.51	1.44	73.01	0.68	8.23	0.06	0.25	2
144	144	1.51709	13.00	3.47	1.79	72.72	0.66	8.18	0.00	0.00	2
145	145	1.51660	12.99	3.18	1.23	72.97	0.58	8.81	0.00	0.24	2
146	146	1.51839	12.85	3.67	1.24	72.57	0.62	8.68	0.00	0.35	2
147	147	1.51769	13.65	3.66	1.11	72.77	0.11	8.60	0.00	0.00	3
148	148	1.51610	13.33	3.53	1.34	72.67	0.56	8.33	0.00	0.00	3
149	149	1.51670	13.24	3.57	1.38	72.70	0.56	8.44	0.00	0.10	3
150	150	1.51643	12.16	3.52	1.35	72.89	0.57	8.53	0.00	0.00	3
151	151	1.51665	13.14	3.45	1.76	72.48	0.60	8.38	0.00	0.17	3
152	152	1.52127	14.32	3.90	0.83	71.50	0.00	9.49	0.00	0.00	3
153	153	1.51779	13.64	3.65	0.65	73.00	0.06	8.93	0.00	0.00	3
154	154	1.51610	13.42	3.40	1.22	72.69	0.59	8.32	0.00	0.00	3
155	155	1.51694	12.86	3.58	1.31	72.61	0.61	8.79	0.00	0.00	3
156	156	1.51646	13.04	3.40	1.26	73.01	0.52	8.58	0.00	0.00	3
157	157	1.51655	13.41	3.39	1.28	72.64	0.52	8.65	0.00	0.00	3
158	158	1.52121	14.03	3.76	0.58	71.79	0.11	9.65	0.00	0.00	3
159	159	1.51776	13.53	3.41	1.52	72.04	0.58	8.79	0.00	0.00	3
160	160	1.51796	13.50	3.36	1.63	71.94	0.57	8.81	0.00	0.09	3
161	161	1.51832	13.33	3.34	1.54	72.14	0.56	8.99	0.00	0.00	3
162	162	1.51934	13.64	3.54	0.75	72.65	0.16	8.89	0.15	0.24	3
163	163	1.52211	14.19	3.78	0.91	71.36	0.23	9.14	0.00	0.37	3
164	164	1.51514	14.01	2.68	3.50	69.89	1.68	5.87	2.20	0.00	5
165	165	1.51915	12.73	1.85	1.86	72.69	0.60	10.09	0.00	0.00	5
166	166	1.52171	11.56	1.88	1.56	72.86	0.47	11.41	0.00	0.00	5
167	167	1.52151	11.03	1.71	1.56	73.44	0.58	11.62	0.00	0.00	5
168	168	1.51969	12.64	0.00	1.65	73.75	0.38	11.53	0.00	0.00	5
169	169	1.51666	12.86	0.00	1.83	73.88	0.97	10.17	0.00	0.00	5
170	170	1.51994	13.27	0.00	1.76	73.03	0.47	11.32	0.00	0.00	5
171	171	1.52369	13.44	0.00	1.58	72.22	0.32	12.24	0.00	0.00	5
172	172	1.51316	13.02	0.00	3.04	70.48	6.21	6.96	0.00	0.00	5
173	173	1.51321	13.00	0.00	3.02	70.70	6.21	6.93	0.00	0.00	5

Fig.1.7 Orange tool datasets

2.8. Information about missing values or outliers:

In this dataset, the features contain no missing values.

With Orange tool:

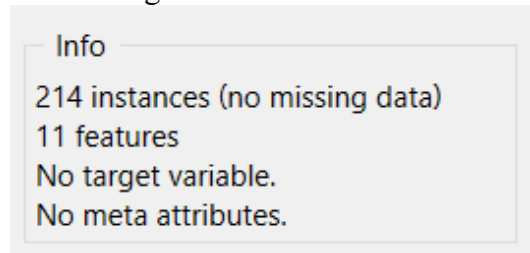


Fig.1.8 Information

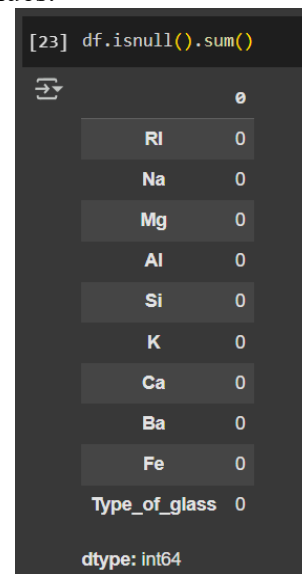


Fig.1.9 Missing values from Python code

With Python:

There's no need to handle missing data in this case, but we do need to address the presence of outlier values.

To identify potential outliers, I first used boxplots in Python for initial inspection.

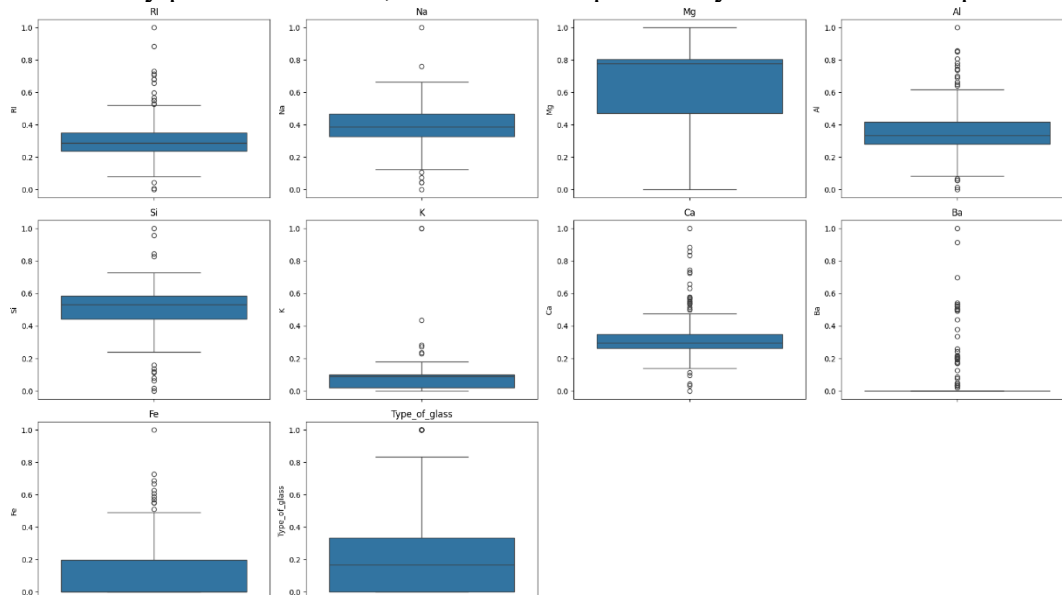


Fig.1.10 Box Plots in Python

Mostly with outliers:

- RI (Refractive Index): Outliers are observed below approximately 0.1, and mostly above 0.6.
- Na (Sodium): Outliers are primarily found below ~0.1, with a few also appearing above ~0.6.

- Al (Aluminum): Most outliers are concentrated above ~ 0.6 , with a few also appearing below ~ 0.1 .
- Si (Silicon): Outliers are present both below ~ 0.2 and above ~ 0.7 .
- Ca (Calcium): Outliers appear below ~ 0.15 and are more frequently above ~ 0.55 .
- Ba (Barium): Most values are clustered around 0.00, considered outliers due to high concentration. Several extreme outliers are observed above ~ 0.4 , reaching up to ~ 0.95 .
- Fe (Iron): Most outliers are found above ~ 0.4 , with one reaching approximately 1.0.
- K (Potassium): A few outliers are present above ~ 0.3 , with one value close to 1.0.

Nearly no outliers:

- Mg (Magnesium): The distribution appears smooth, with no visible outliers; all values fall within the range of approximately 0.05 to 0.85.

Within the Orange tool, for handling them, I applied the Outliers widget and selected the Isolation Forest method with an 5% threshold (%) to avoid excessive removal of data — a choice that offers a good balance between retaining useful data and filtering out anomalies, as forests are generally robust to noise and outliers. Because of the limited data size, it is important to be cautious when removing outliers, in order to retain as much valuable information as possible.

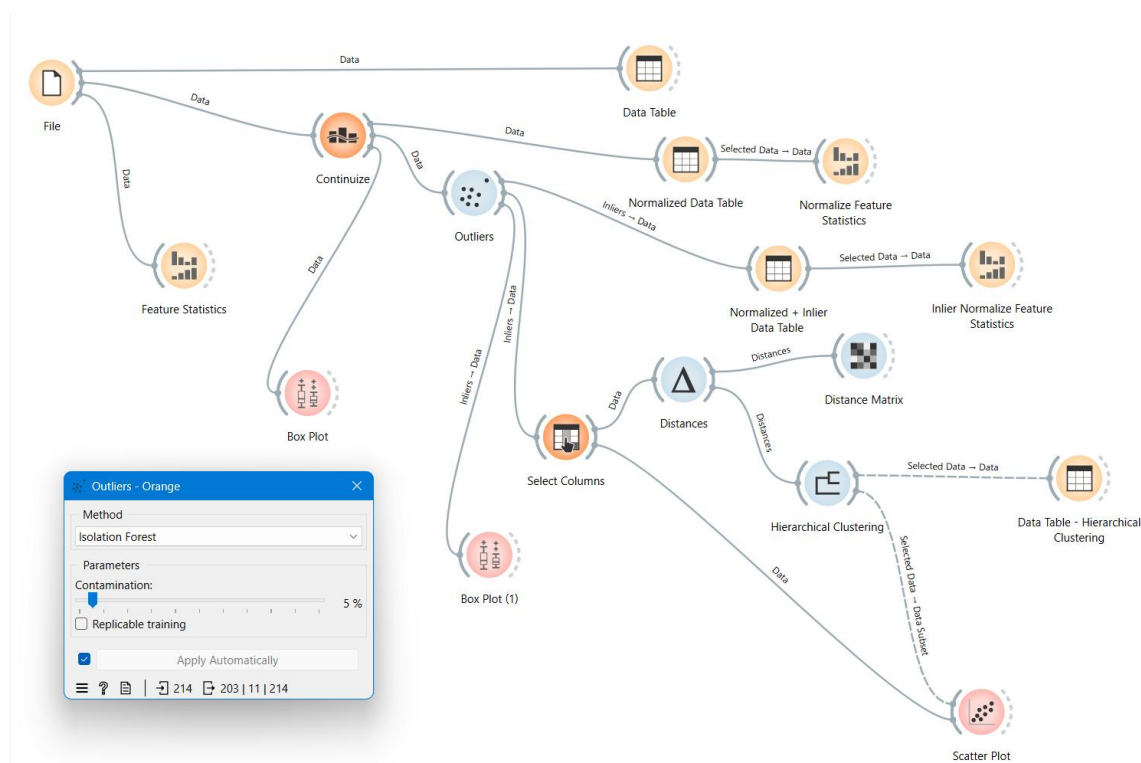


Fig.1.11 Random Forest (5%) in Orange Tool

3. Visual and statistical representation of the dataset

3.1. Line Plots:

Representing feature value:



Fig.1.12 Line Plots - Python

More specific, we will see the similarity of the trends between RI (Refractive Index) and Ca (Canxi):

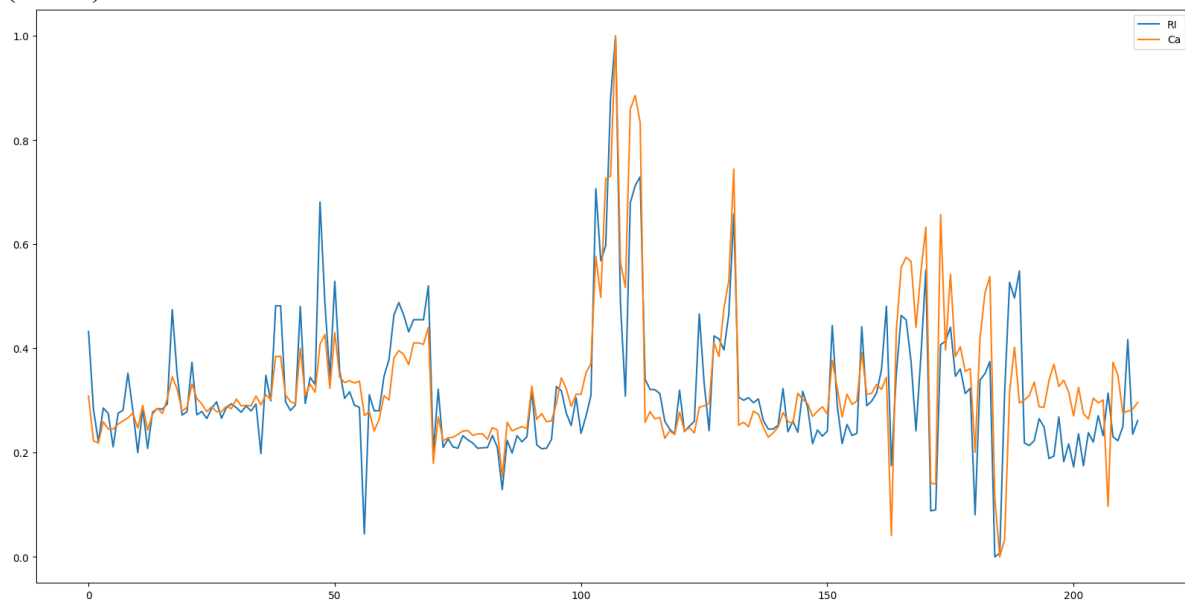


Fig.1.13 Line Plots (RI vs Ca) - Python

3.2. Scatter Plots:
Ca vs Si before the outlier process:

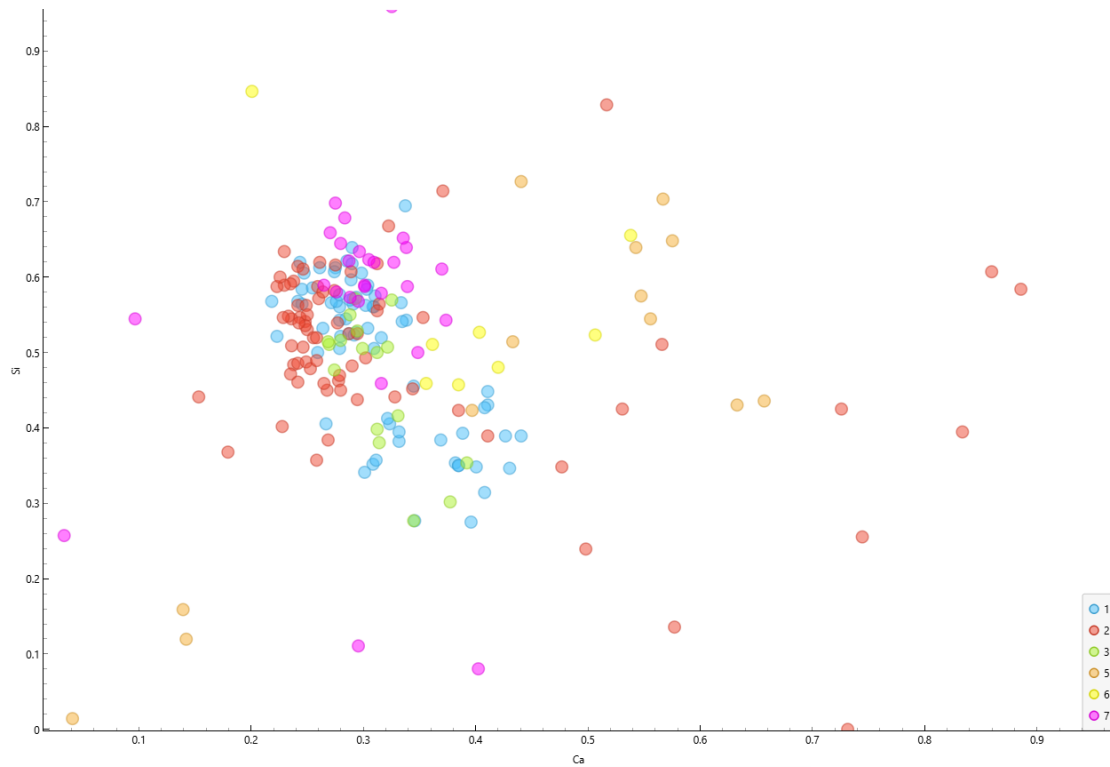


Fig.1.14 Scatter Plots (Ca vs Si)

Mg vs RI before outlier process:

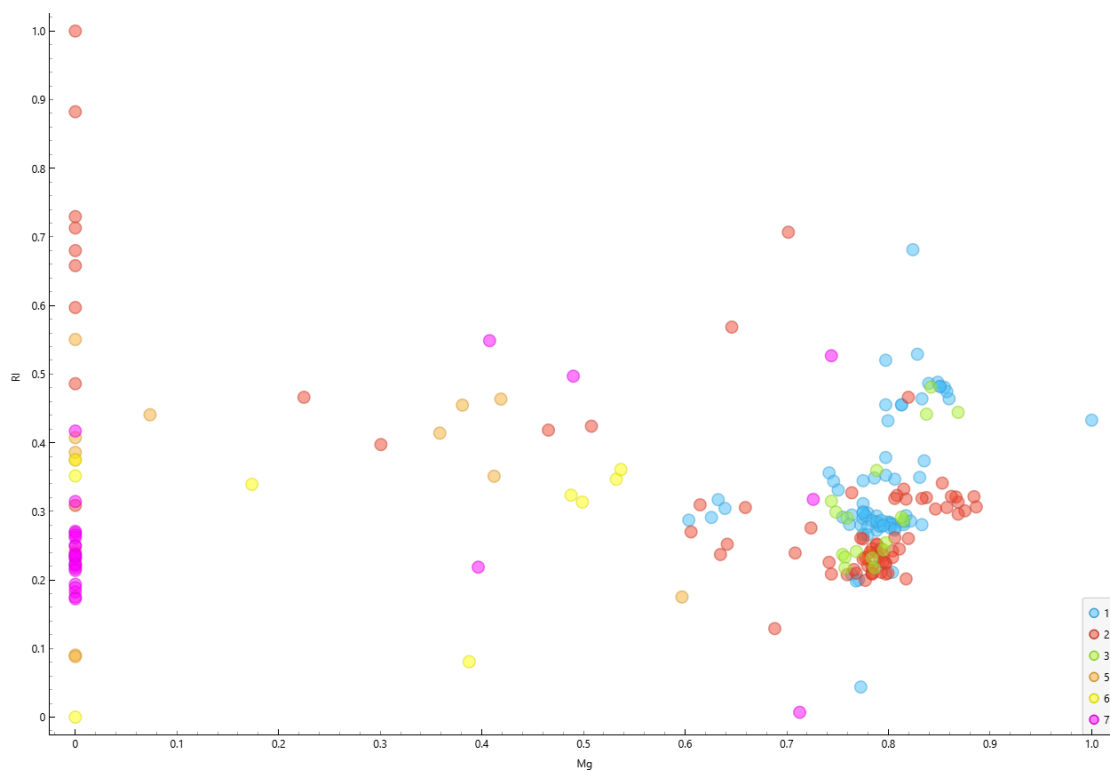


Fig.1.15 Scatter plots in Orange tool (Mg vs RI)

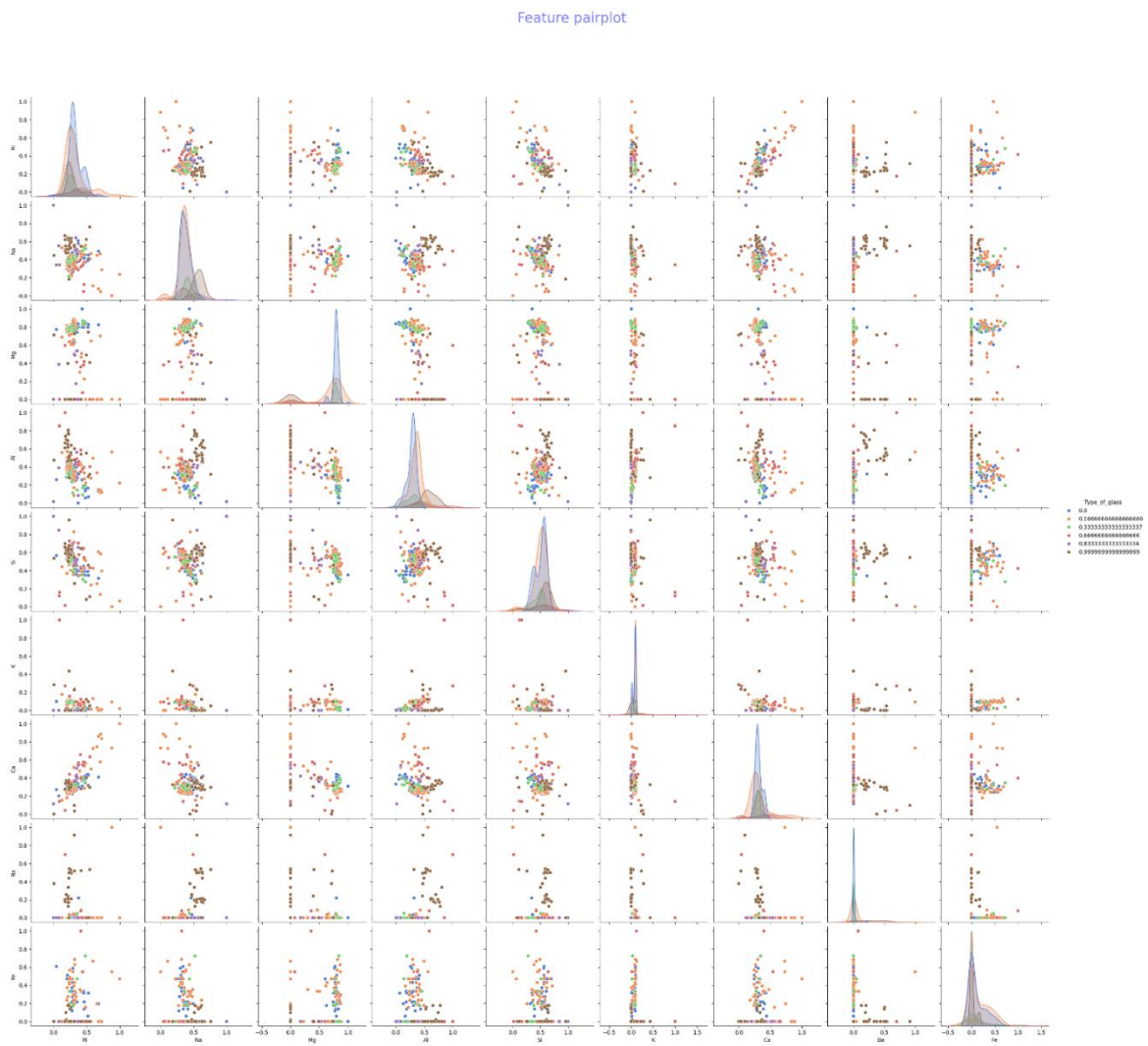


Fig.1.16 Scatter Plots for all Elements

3.3. Histograms:

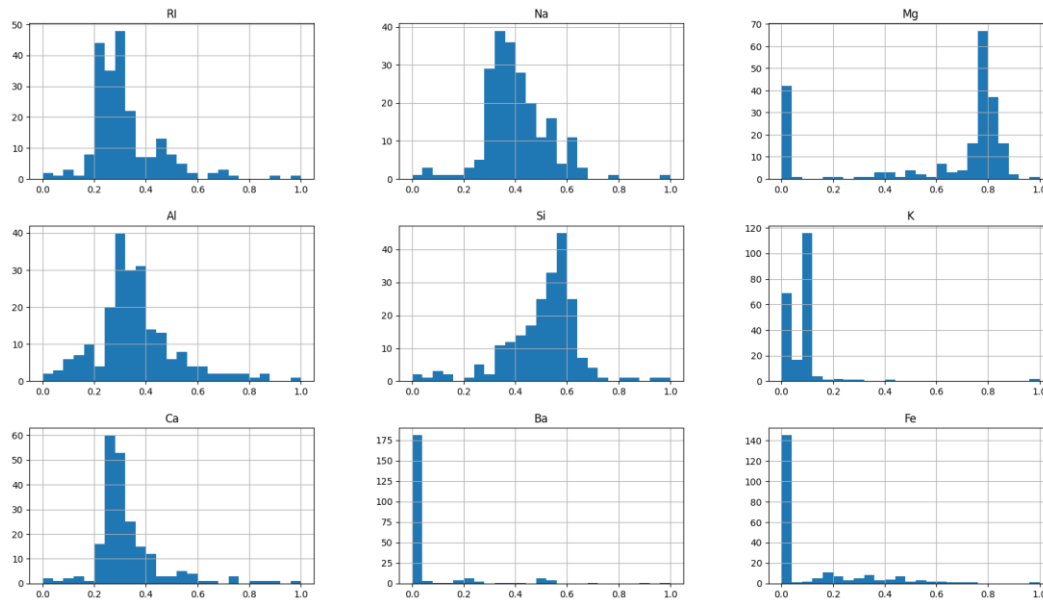


Fig.1.17 Histograms

For both histogram + Normal Distribution:

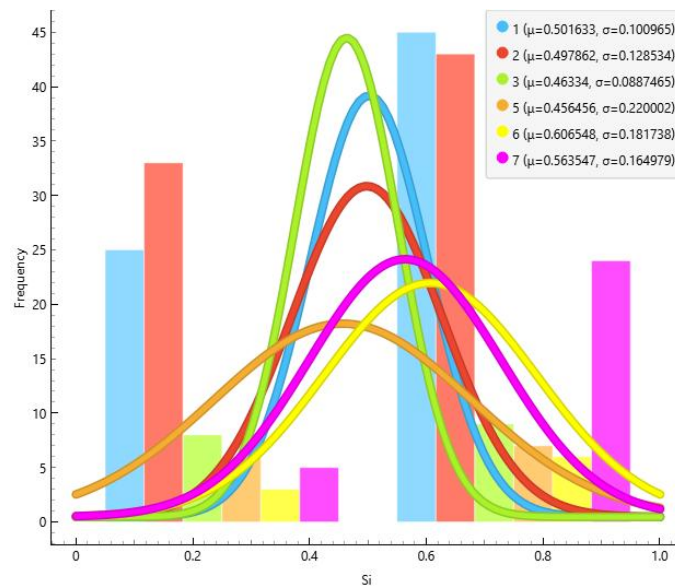


Fig.1.18 Histogram and Normal Distribution

Histograms which are only for classes:

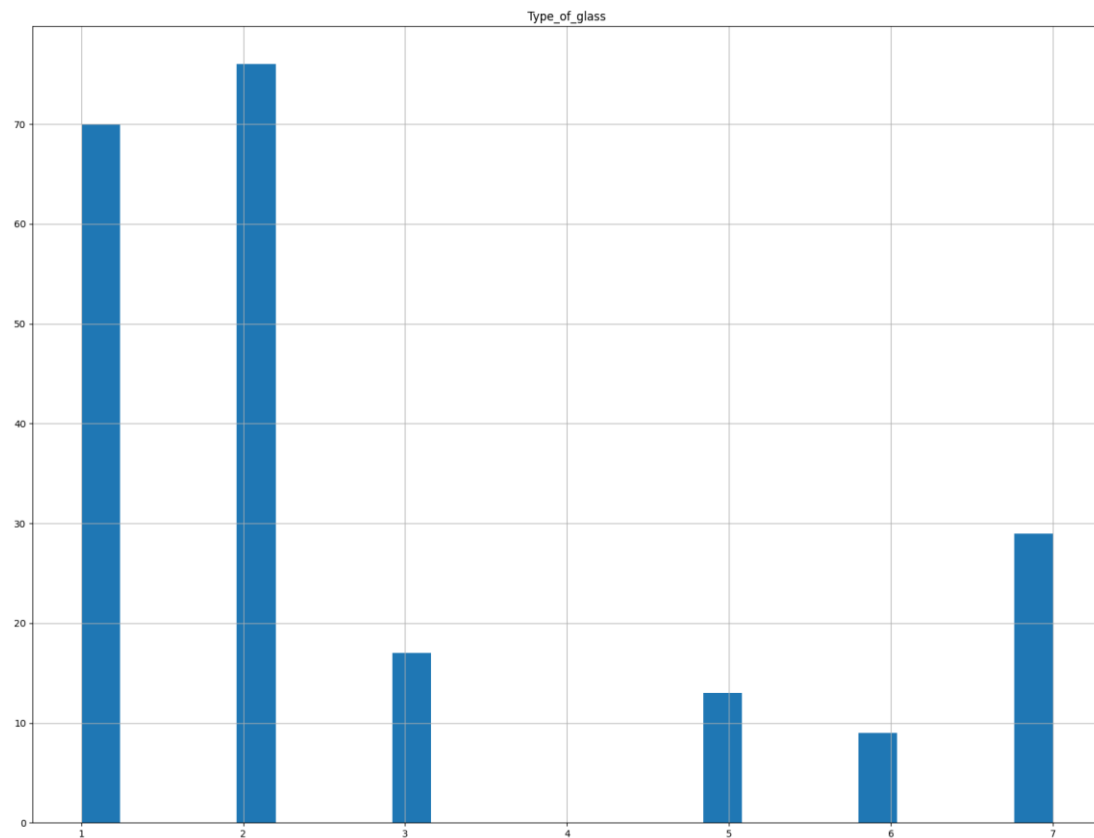


Fig.1.19 Histogram only for classes – Type_of_glass

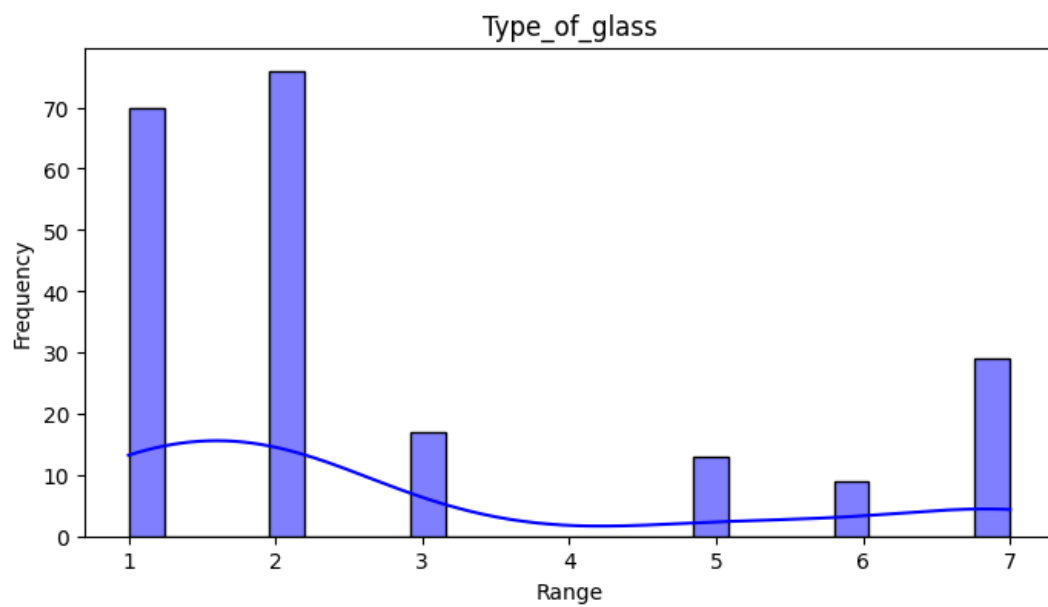


Fig.1.20 Histogram with KDE line - Types_of_glass

3.4. Box plots:

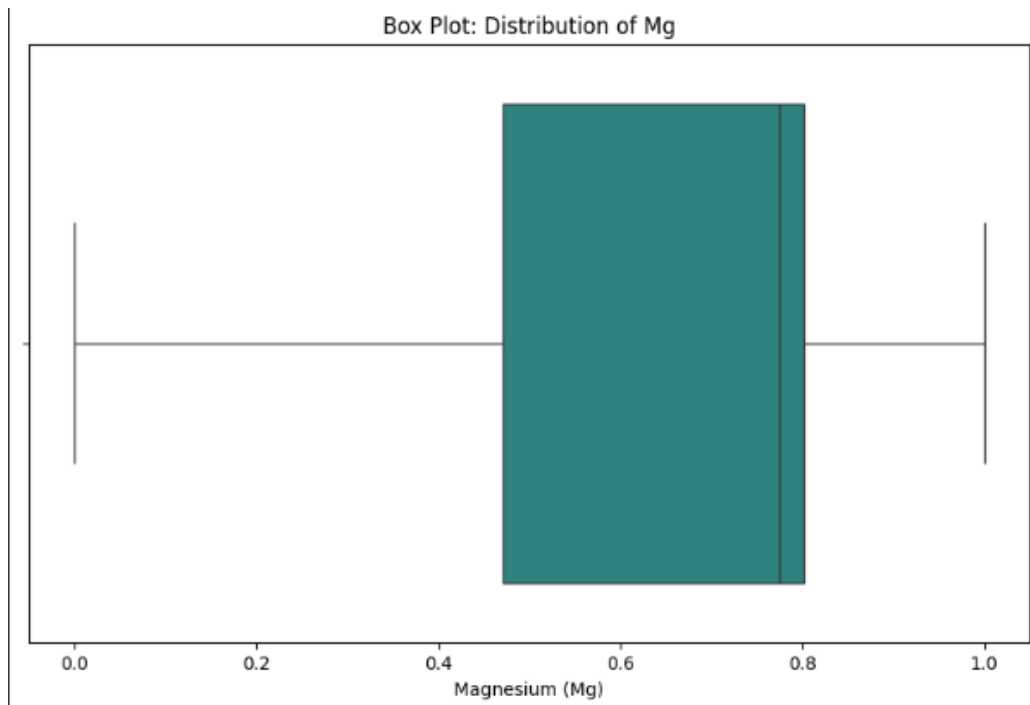


Fig.1.21 Box Plots for Mg

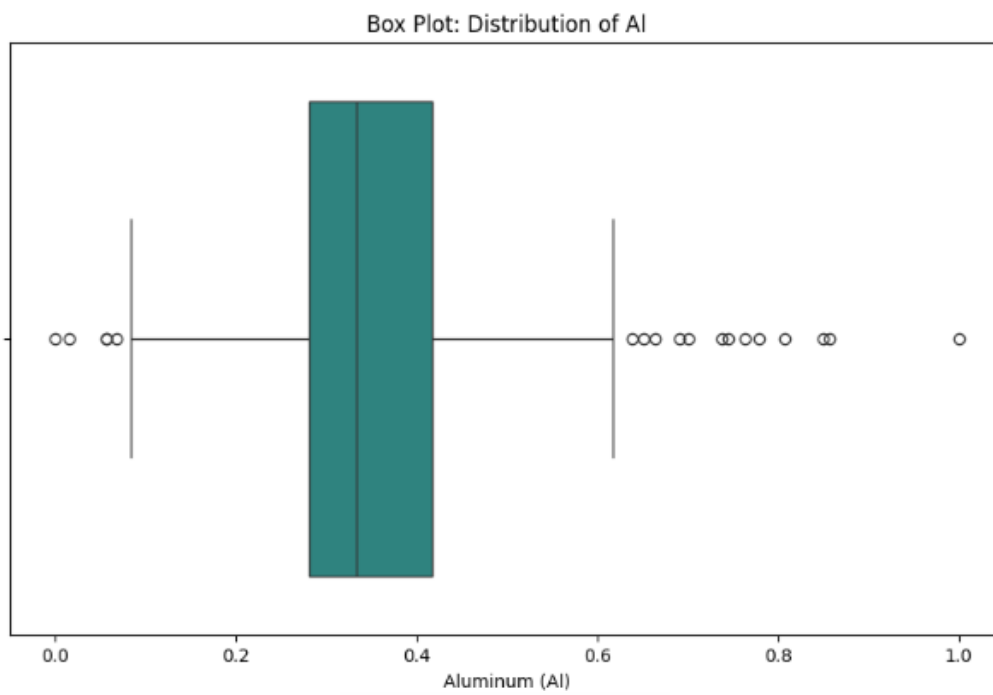


Fig.1.22 Box Plots for Al

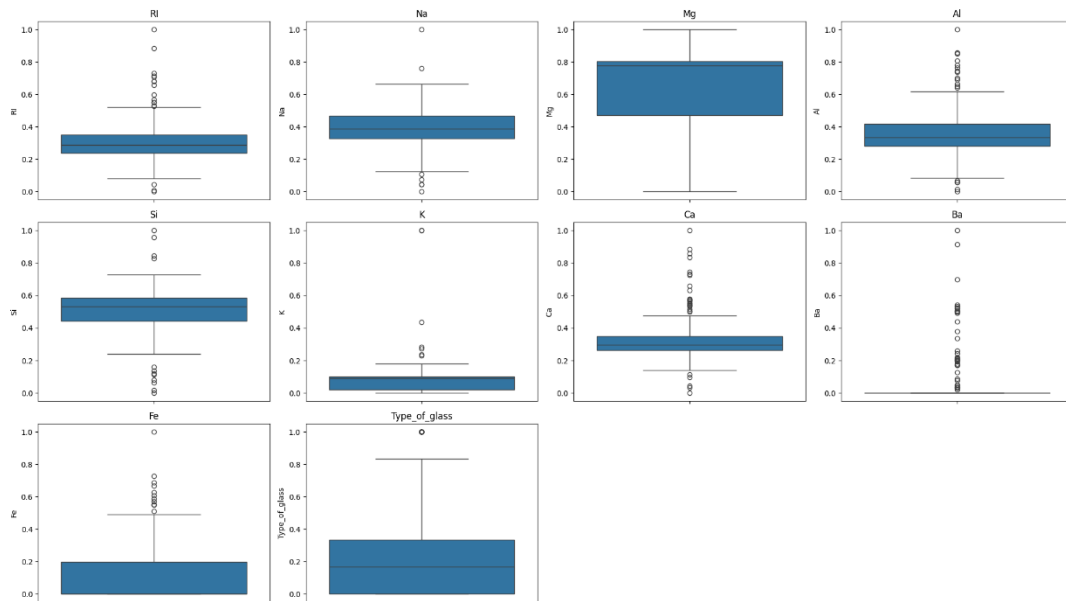


Fig.1.23 Box Plots for All Features including Type_of_glass

3.5. Statistical information:

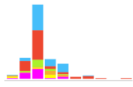
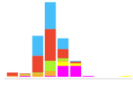
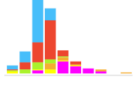
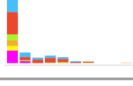
	Name	Distribution	Mean	Mode	Median	Dispersion	Min.	Max.	Missing
N	RI		0.316744	0.208516	0.286655	0.419901	0.00	1	0 (0 %)
N	Na		0.402684	0.341353	0.386466	0.304234	0.00	1	0 (0 %)
N	Mg		0.597891	0.00	0.775056	0.536046	0.00	1	0 (0 %)
N	Al		0.359784	0.389408	0.333333	0.431292	0.00	1	0 (0 %)
N	Si		0.50731	0.544643	0.532143	0.272	0.00	1	0 (0 %)
N	K		0.0800412	0.00	0.089372	1.30904	0.00	1	0 (0 %)
N	Ca		0.327785	0.241636	0.29461	0.402563	0.00	1	0 (0 %)
N	Ba		0.0555704	0.00	0.00	2.83385	0.00	1	0 (0 %)
N	Fe		0.111783	0.00	0.00	1.70517	0.00	1	0 (0 %)
C	Type_of_glass			2		1.51			0 (0 %)

Fig.1.24 Statistical information

working_df.describe()										
	RI	Na	Mg	Al	Si	K	Ca	Ba	Fe	Type_of_glass
count	214.000000	214.000000	214.000000	214.000000	214.000000	214.000000	214.000000	214.000000	214.000000	214.000000
mean	0.316744	0.402684	0.597891	0.359784	0.507310	0.080041	0.327785	0.055570	0.111783	0.296729
std	0.133313	0.122798	0.321249	0.155536	0.138312	0.105023	0.132263	0.157847	0.191056	0.350623
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.235843	0.327444	0.471047	0.280374	0.441071	0.019726	0.261152	0.000000	0.000000	0.000000
50%	0.286655	0.386466	0.775056	0.333333	0.532143	0.089372	0.294610	0.000000	0.000000	0.166667
75%	0.351514	0.465414	0.801782	0.417445	0.585268	0.098229	0.347816	0.000000	0.196078	0.333333
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

Fig.1.25 Describing the features using Python

3.6. Confusion Matrix:

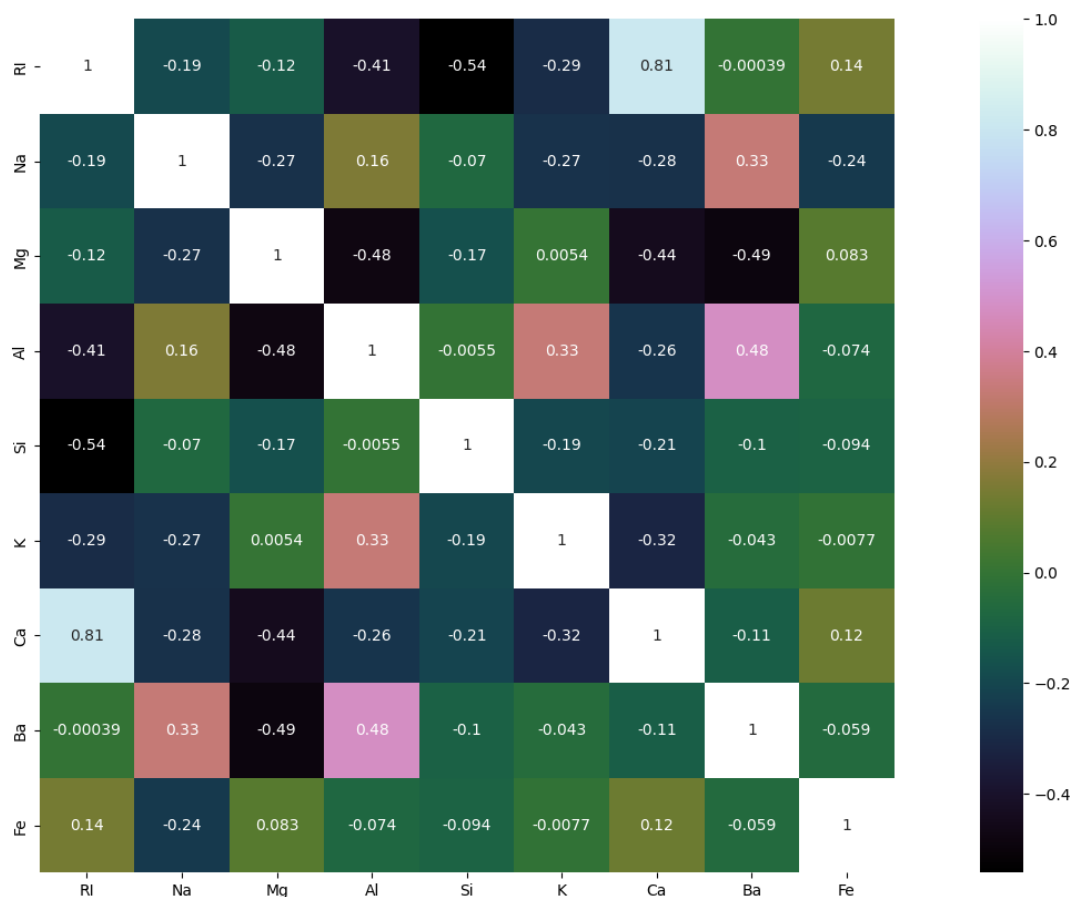


Fig.1.26 Confusion matrix

4. Answers to questions

4.1. Are the classes in the dataset balanced, or does one class (or several classes) prevail?

The classes are **not balanced**. Classes 1 and 2 are the most prevalent, while classes 5 and 6 are the least prevalent.

The class distribution is shown in the bottom-left histogram (Fig 1.17). The histogram for "Type_of_glass" reveals that the dataset is **not balanced**: a few classes have a much higher number of samples, while others have significantly fewer. Specifically, two classes dominate the dataset, while the remaining classes are underrepresented. Class 4 does not appear.

4.2. Does the visual representation of the data allow you to see the structure of the data?

Yes, it allows to see the structure. However, the data structure does not allow for clear separation of different classes (glass types) in most of the plots. The clusters overlap significantly, especially in the scatter plots, and no strong boundary is visible between different class labels (like glass type 1, 2, 3, etc.).

- **Pair plot (Fig.1.16):** This plot shows the relationships between each pair of features, coloured by class. The classes are not clearly separated; there is considerable overlap

among data points from different classes in most feature combinations. This suggests that the data objects belonging to different classes **cannot be easily separated** based on these features.

- **Line Plot (Fig.1.12):** The line plot of feature values across samples shows high variability and overlap among the classes, further indicating that **the classes are not cleanly separable** in the feature space.

4.3. How many data groupings can be identified by studying the visual representation of the data?

Only a few loose groupings, but most classes are merged and not clearly separable.

Roughly 3 to 4 distinguishable data groupings (clusters) can be identified from the visual representations — though they are not perfectly separated. Some clusters overlap significantly, which makes it hard to confidently identify all 7 glass types (as labelled) just by visual inspection.

Evidence:

- In the scatter plots (e.g. Ca vs Si, Mg vs RI), we can see a dense cluster in the middle, and a few smaller, scattered groups elsewhere (**Fig.1.14 & Fig.1.15**).
- The pair plot confirms this: only a few groups **show partial separation** while others heavily overlap. Most data points from different classes are merged, and no distinct clusters corresponding to each class are visible (**Fig.1.16**).
- The histograms and boxplots also show most features have **a wide range with overlapping** distributions. Especially, the histograms of individual features (**Fig.1.17**) also show that the distributions for most features overlap significantly across classes, supporting the observation that clear, separable groupings are not present (**Fig.1.19 & Fig.1.23**).

4.4. Are the identified data groupings close to each other or far from each other?

Groupings are close together; there are no clear boundaries between classes. The identified data groupings are mostly close to each other, with significant overlap in many cases which we can observe through scatter plots. Only a few points or small clusters appear farther apart, but many of the groupings are densely packed and **not well separated**.

5. Conclusions arising from the analysis of statistical indicators

From **histograms and statistical information (mean, median, mode)**, we observe that:

- RI with Mean: 0.3167, Median: 0.2867 → Slight right skew
- Na, Al, Si all show values where mean $\approx$ median $\approx$ mode → Roughly symmetric distributions → Good for training.
- Si has a well-shaped bell curve (normal-like), making it potentially more useful in modeling.
- Mg with Mean: 0.5979, Median: 0.7751, Mode: 0 → Highly left-skewed, many zeros.
- Ba, Fe, K: All have mode = 0 and low median → Most values are clustered at zero, indicating sparse features. → Need to remove outliers or drop these features.

From the statistical information (dispersion), we observe that:

- Ba: 2.83, the highest → values are either 0 or very spread out.
- Fe: 1.705 → also widely varied with many outliers.

From Boxplots:

- Outliers are frequent in Ba, Fe, and K, seen as many points as possible outside whiskers.
- Mg shows a compact distribution but is skewed — very few values close to 0, many close to 1.
- RI, Na, Al, Ca have mild outliers but are more normally distributed.

From Confusion Matrix:

- The strongest positive correlation is between RI (Refractive Index) and Ca (Calcium), with a coefficient of 1 & 0.81. This suggests that these two features carry similar information, and including both in further analysis may introduce redundancy.
- Most other feature pairs have weak or negligible correlations (coefficients close to 0), indicating that they provide unique information.

Part II

1. Hierarchical clustering

1.1. Hyperparameters available in the Orange tool:

Hyperparameter	Description
Linkage	Determines how the distance between clusters is calculated when merging them. Common methods include single, complete, average, etc. Average Linkage defines the distance between two clusters as the average of the distances between all possible pairs of points. This typically produces clusters that are moderately compact.
Distance	Specifies the metric used to measure the distance between data points (e.g., Euclidean, Manhattan, Cosine). Only Euclidean is compatible with Ward linkage. Not Euclidean (normalized) – because we had normalized it already.
Manual/ Height Ratio/ Top N	Controls how the dendrogram is cut to form clusters. We can either manually select a level, specify a height ratio (percentage of dendrogram height), or choose the top N clusters. We specified it with option - manual (Due to the easiness of changing the cut-off line and visualizing).
Pruning (None/ Max depth)	Limits the depth of the dendrogram tree to control the level of clustering granularity. Useful for reducing noise or focusing on higher-level clusters. We chose None because at the beginning we wanted to explore the original data, to determine a reasonable height threshold for clustering (height ratio), to see specifically which cluster each point belongs to.

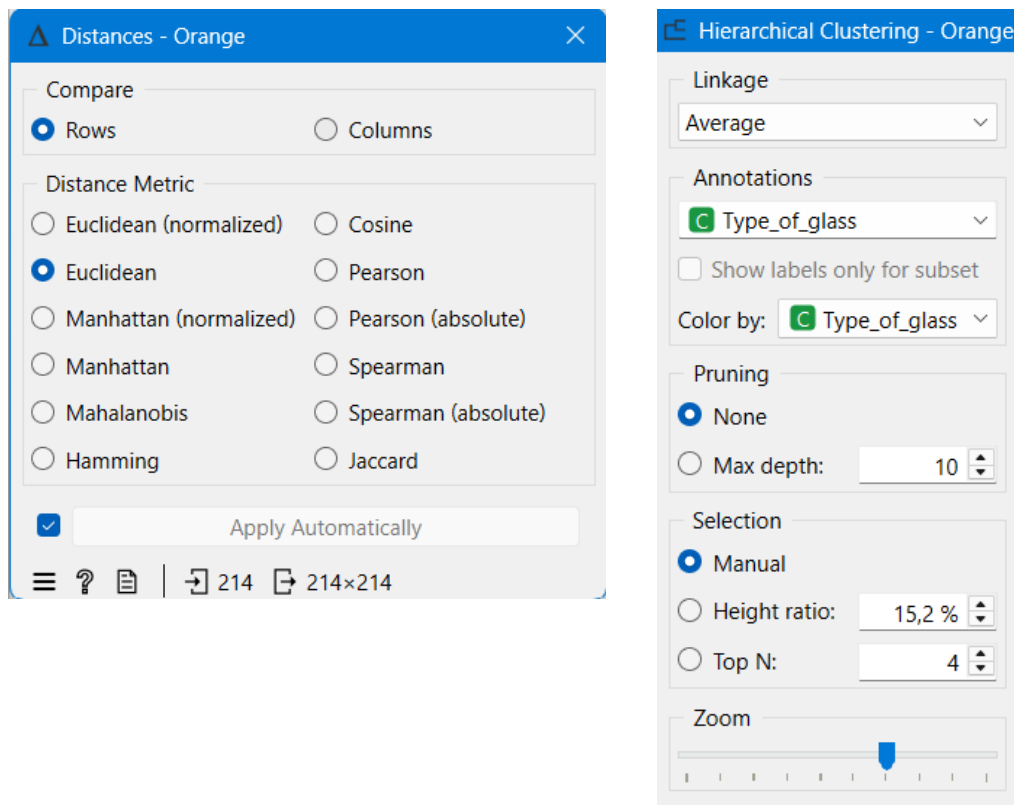


Fig 2.1 Hyperparameter – distance value set for the algorithm & Hierarchical clustering parameters

At the bottom of the dendrogram, there are many short branches, showing that similar data points are grouped first. As moving up, these small groups merge progressively, forming larger clusters — this suggests the data has a natural hierarchical structure. There's no chaining effect or erratic long connections, which is a good sign of well-structured grouping. Near the top of the dendrogram, a few branches split off early and stay separated for a long time before merging. This usually points to outliers or very distinct subgroups that are different from the rest of the data.

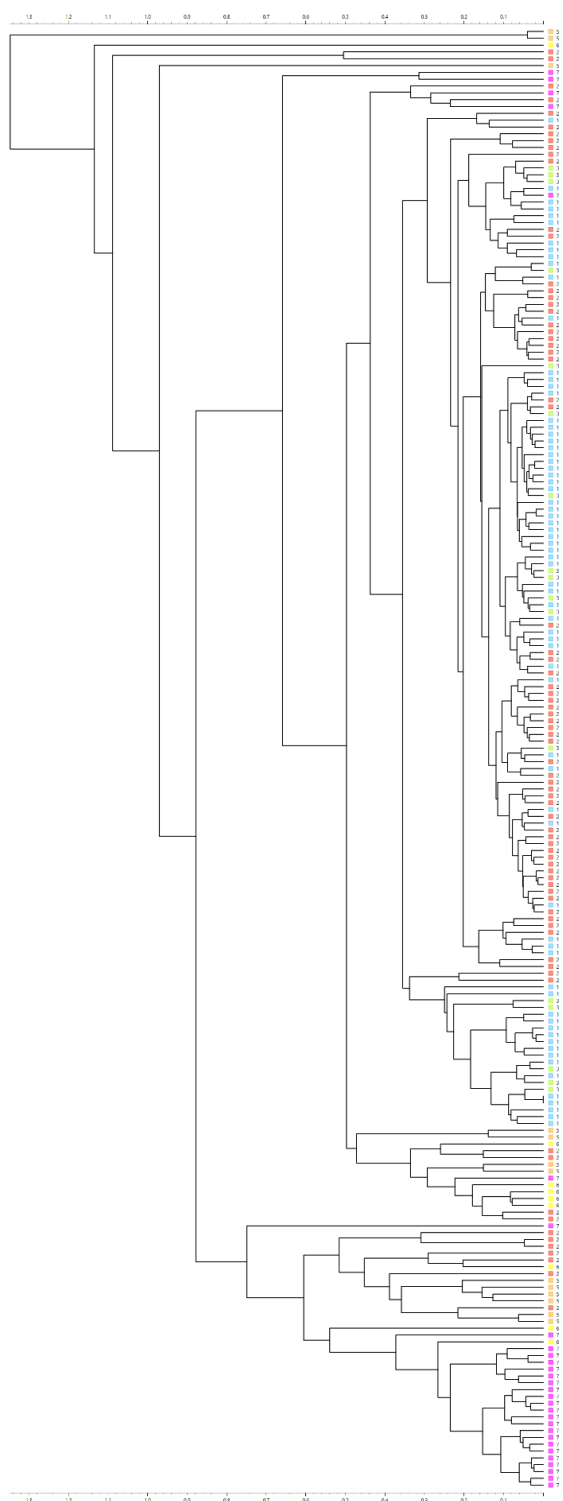


Figure 2.2 Dendrogram with Average Linkage

1.2. Description of experiments

Experiment 1 with a certain position of the horizontal cut offline at 1.0

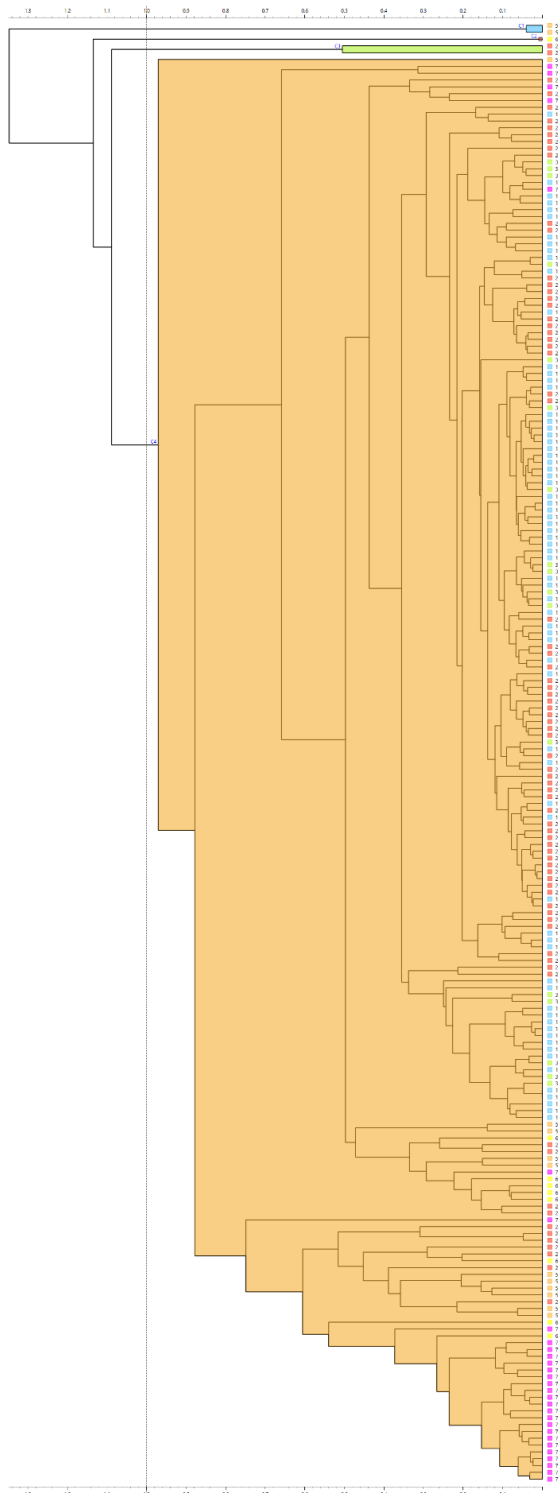


Fig.2.3 *cutoff line at 1.0*

When the cut-off is set at 1.0, the dendrogram splits into about 4 large clusters. This provides a general overview and reveals the main structure of the data. However, these clusters are quite broad, meaning that some data points that are actually different may end up in the same group.

Experiment 2 with a certain position of the horizontal cut off line at 0.7

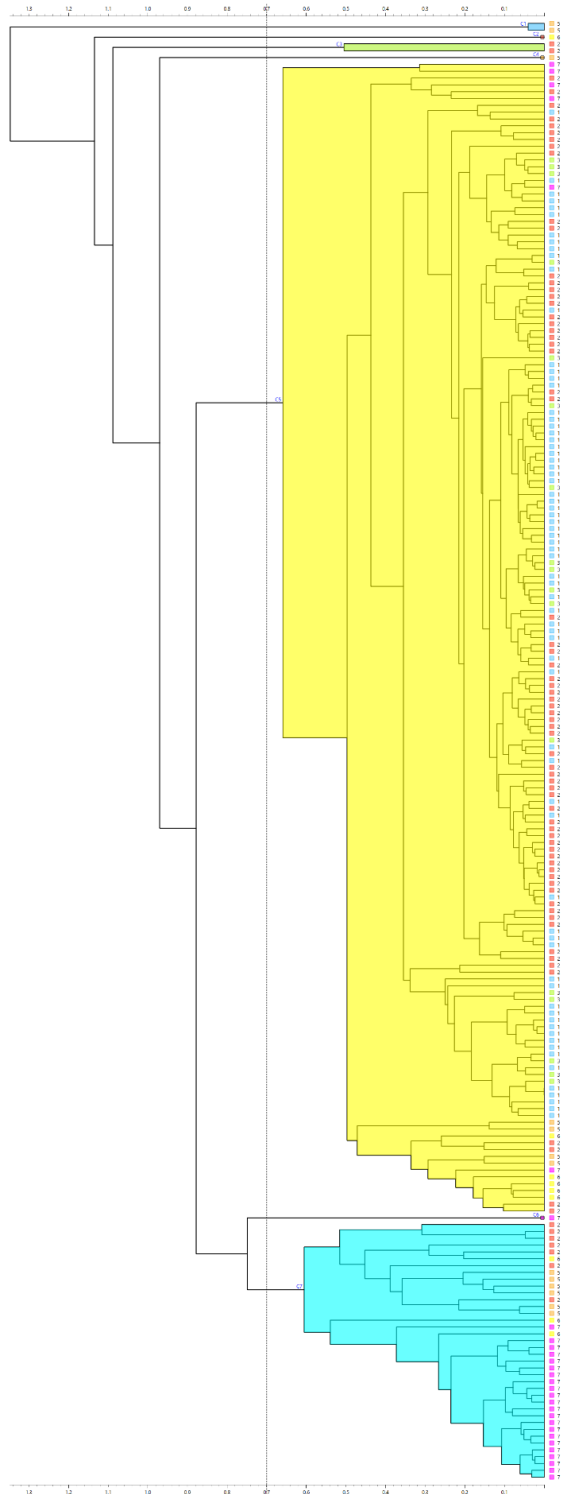


Fig.2.4 *cutoff line at 0.7*

Lowering the cut-off to 0.7 results in better-defined and more detailed clusters, producing approximately 7 clearer and more consistent groups. This level strikes a nice balance, it's not overly simplified, yet still easy to interpret. However, some clusters still contain mixed data points from different original classes. For example, cluster 7 mostly contains data points from class 7, but also includes points from classes 2, 5, and 6.

Experiment 3 with a certain position of the horizontal cut offline at 0.5

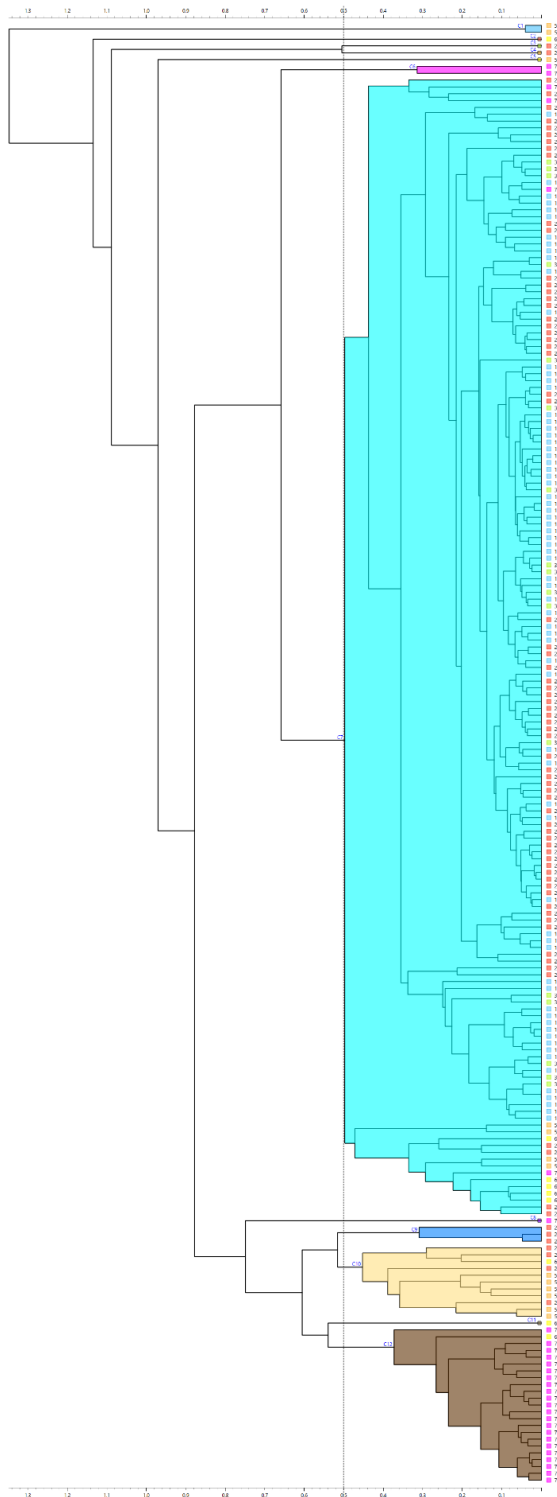


Fig.2.5 cutoff line at 0.5

Setting the cut-off to 0.5 divides the dendrogram into about 12 smaller clusters. At this level, the data points within each group are highly similar to one another. Visually and theoretically, this seems to be the most optimal choice. For example, class 7 becomes more distinctly represented in cluster 12, although it still includes one data point from class 6. These clusters group similar points effectively. This seems to reflect the natural segmentation of the data without over fragmenting.

Experiment 4 with a certain position of the horizontal cut off line at 0.24

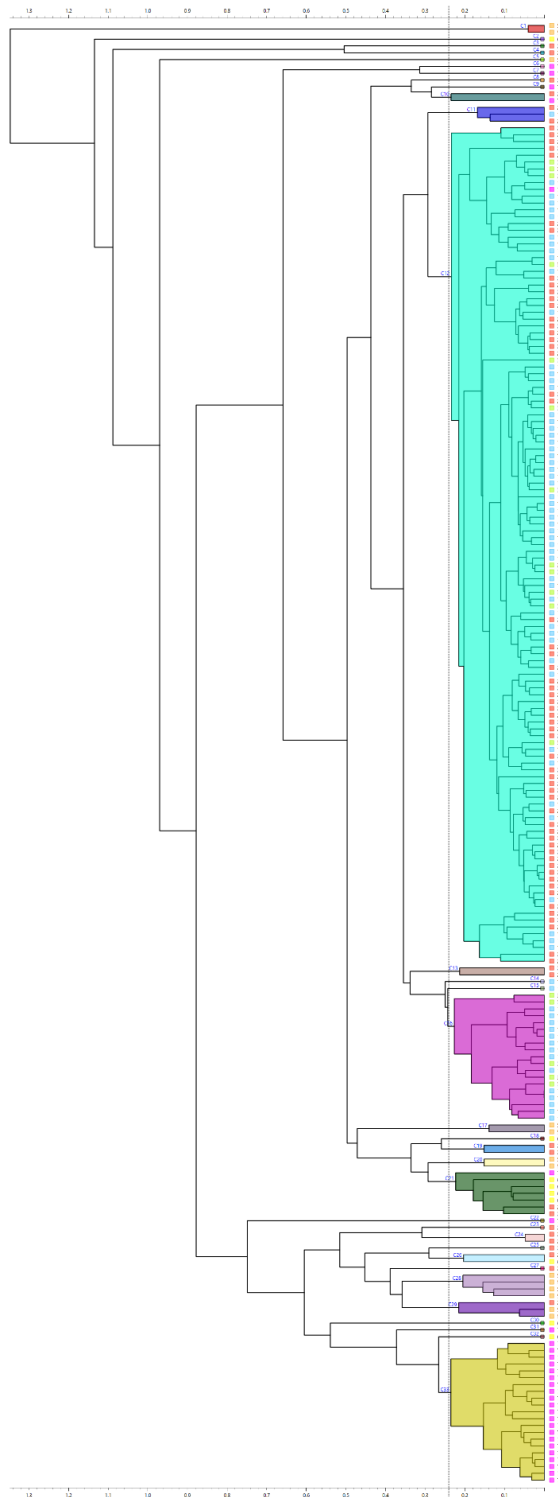


Fig.2.6 *cutoff line at 0.24*

At a cut-off of 0.24, the dendrogram breaks into 33 very small clusters. This reveals fine-grained details, which might be useful for tasks requiring high precision. However, the downside is that clustering becomes harder to interpret. The large number of clusters can overwhelm the broader structure of the data. Over-segmentation occurs - minor differences between data points lead to the creation of new clusters.

1.3. Conclusions from experiments:

By changing the horizontal cut-off values in the dendrogram using Average Linkage, we saw how the number and size of clusters affect how easy the results are to understand and how accurate they are. A high cut-off (like 1.0) gives a simple view with fewer, larger clusters, which is good for seeing the overall structure but may miss smaller details. When the cut-off is lower (like 0.7 or 0.5), the clusters become clearer and more balanced, showing more natural groupings in the data. Among these, the cut-off at 0.5 seems to work best, giving a good balance between detail and clarity. But if the cut-off is too low (like 0.24), the data is split into too many tiny clusters, which makes the result harder to understand and may lead to overfitting. Therefore, it's important to choose a cut-off level that matches the purpose of the analysis—whether it's for a simple overview or for finding detailed patterns.

2. K-means algorithm

2.1. Hyperparameters available in the orange tool:

Hyperparameter - K	Description
k (Number of Clusters)	Specifies how many clusters the algorithm should try to find in the dataset. At the beginning we chose the option of the interval from 2 to 10 for observing the silhouette scores. After realizing which number will be the highest score – 4, we have chosen fixed with 4 .
Normalize Columns	Scales all features to the same range to prevent biased clustering. We turned it on because K-means is very sensitive to the distance. Without normalization, features with larger values may dominate the clustering process.
Initialization	Defines how the initial cluster centroids are selected (2 options: Initialize with K Means++, Random initialization) We chose to Initialize with K Means++ , because the cluster initialization point selection method is better than random selection and helps reduce the risk of getting stuck in local minima and often gives better results.
Re-runs	Number of times K-means is run with different centroid seeds; best is chosen.
Maximum Iterations	Max number of iterations the algorithm performs before stopping. Avoid the situation where the algorithm runs forever without converging. 1000 is a safe and common value .

2.2. Description of experiments

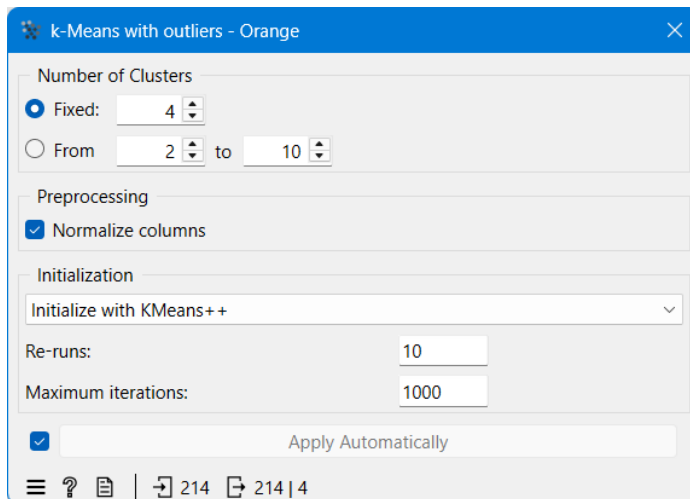


Fig.2.7 K-means parameters (fixed – 4)

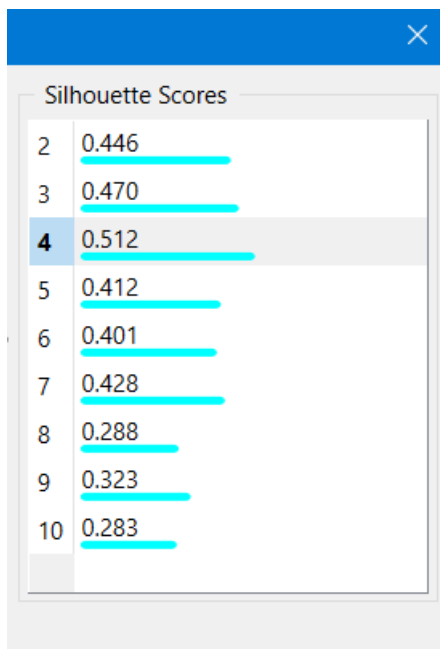


Fig.2.8 Silhouette coefficient with at least 5 different k values

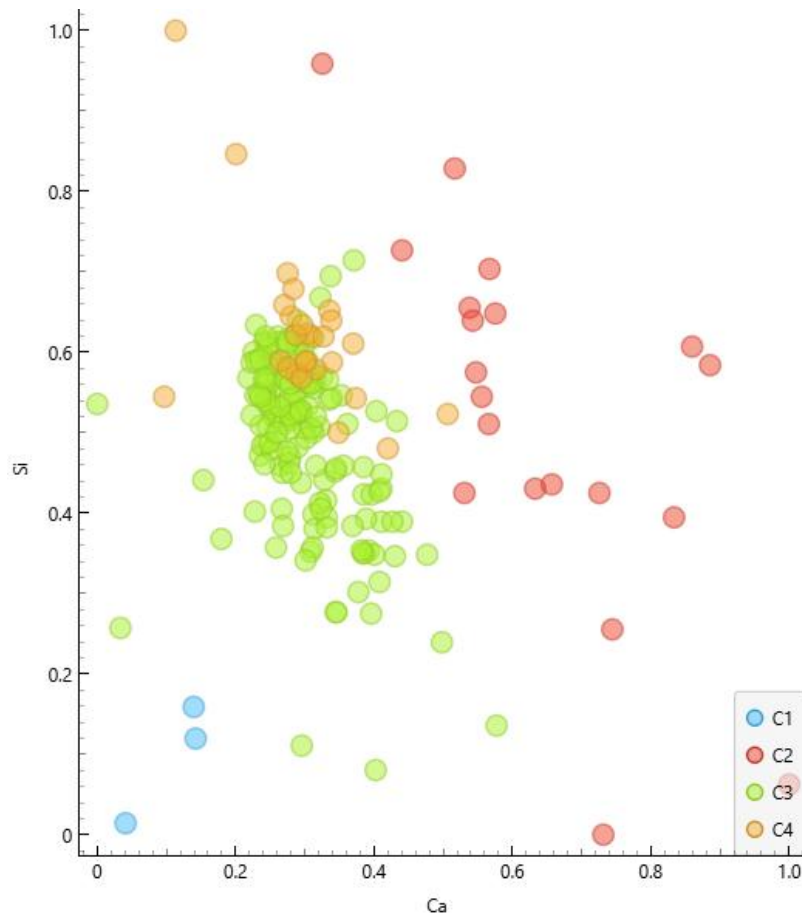


Fig.2.9 Scatter Plots for Ca and Si - according to the best value 4 of the Silhouette coefficient

2.3. Conclusions from experiments:

When the number of clusters varied from 2 to 10, the highest silhouette score achieved was 0.512 at 4 clusters, indicating that this is the most appropriate number of clusters for the given data. This is also confirmed in the first screenshot, where the number of clusters was fixed at 4.

The silhouette score ranges from -1 to 1, where a higher value indicates that the data points are well matched to their own cluster and poorly matched to neighboring clusters. A score of 0.512 suggests that the clustering structure is moderately well-defined, although there may still be some overlap between clusters.

Based on Fig 2.10 - Cluster Distribution in Feature Space:

The 2D scatter plot using Ca and Si features shows that:

- Cluster C3 (green) dominates the dataset and is densely packed in the central area.
- Cluster C2 (red) is clearly separated on the right side, indicating good distinction from C3.
- Cluster C4 (orange) slightly overlaps with C3, suggesting that these two clusters are not fully separable.
- Cluster C1 (blue) contains very few points scattered far from the main groups, indicating they may be outliers.

The classes in the dataset are relatively separable, with the optimal number of clusters being 4. There is still some overlap between clusters—particularly between C3 and C4 - but the overall structure is reasonably clear. Cluster C1 likely represents outlier points or noise.

Final conclusions

The dataset is not perfectly separable. There is significant overlap between classes, especially due to imbalanced class distribution and overlapping feature values.

However, both algorithms identified moderate separability, especially under optimized conditions:

- Ward's linkage at cut-off = 0.5 with 12 clusters
- K-Means with $k = 4$ with 4 clusters

These findings suggest that while the glass types do share overlapping chemical properties, some clusters and glass types can still be partially distinguished, particularly in reduced feature spaces like Ca–Si.

Part III

1. Description of the selected algorithms

1.1. Title of the first algorithm:

Random Forest

1.2. Description of the first algorithm:

Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problem in Machine Learning. It is based on the concept of ensemble learning, which involves combining the outputs of multiple individual models (in this case, decision trees) to produce a more accurate and robust overall prediction.

According to GeeksforGeeks – “**Random Forest Algorithm in Machine Learning**”, the Random Forest algorithm operates through several key steps:

- It constructs multiple decision trees by drawing random samples from the original dataset. Each tree is trained on a different subset, introducing variability and reducing correlation among trees.
- For each tree, the algorithm randomly selects a subset of features at each split rather than using all available features. This random feature selection increases diversity across the trees.
- Once all trees are trained, each one makes an individual prediction based on the data it has seen.
- For classification problems, the final output is determined by majority voting — the class that receives the most votes from the individual trees becomes the final prediction.
- The randomness in both data sampling and feature selection helps to reduce overfitting, leading to more accurate and robust predictions.

According to GeeksforGeeks – “**Random Forest Algorithm in Machine Learning**”, key characteristics of Random Forest include:

- Independence among trees: Each tree in the forest makes its own decision independently, which helps avoid bias and correlation between models.
- Diversity through randomness: Random selection of data and features ensures that trees learn different patterns, reducing the risk of systematic error.
- Ensemble prediction: For classification tasks, the result is based on majority voting among all decision trees. For regression tasks, the final output is the average of all tree predictions.
- Feature importance evaluation: The algorithm ranks features based on their contribution to the prediction, offering useful insights for feature selection and interpretability.
- Flexibility and reliability: Random Forest performs well on a wide range of datasets and requires minimal tuning. It is robust to outliers and noise in the data.

1.3. Title of the second algorithm:

Logistic Regression

1.4. Description of the second algorithm:

Logistic Regression is primarily employed for classification tasks, particularly binary classification. It predicts one of two possible classes for a given data point—such as success/failure, good/bad, yes/no, or win/loss.

Based on the type of dependent variable, Logistic Regression can be categorized into three forms:

- Binomial Logistic Regression is used when the dependent variable has exactly two possible outcomes.
- Multinomial Logistic Regression applies when the dependent variable can take on three or more distinct categories that do not follow any particular order.
- Ordinal Logistic Regression is used when the dependent variable has three or more categories that follow a logical order.

Assumptions of Logistic Regression:

- No Outliers: The model assumes that the dataset does not contain outliers. Extreme values or influential data points should be minimized or removed because they can distort the results significantly. For the Glass Identification dataset, it's important to detect and manage outliers, as well as normalize the features to ensure the model behaves consistently.
- No Multicollinearity: Independent variables should not be highly correlated with one another. Multicollinearity can affect both the interpretability and the performance of the model. If any pair shows high correlation (e.g., greater than 0.8), we should consider removing or combining those variables.
- Mutually Exclusive Target Classes: The dependent variable (target) must consist of distinct and non-overlapping categories that collectively account for all possibilities. In our case, the target variable is the Glass Type, with values such as 1, 2, 3, 5, 6, and 7. Each sample belongs to only one category; thus this assumption is satisfied.

2. Description of hyperparameters

Hyperparameter	Description and values
Artificial Neural Networks	
Neurons in Hidden Layers	Defines the number of neurons (units) in each hidden layer of the neural network. We experimented with different architectures (e.g., single vs. multiple hidden layers) and chose a balanced structure (e.g., 64, 128) to balance model complexity and performance. Too few neurons may lead to underfitting, while too many may cause overfitting.
Activation Function	Determines the non-linear transformation applied to the output of each neuron. Common choices include ReLU, Sigmoid, Identity and Tanh. We used ReLU (Rectified Linear Unit) because it helps mitigate the vanishing gradient problem, speeds up training, and generally performs well in deep networks.

Solver	Specifies the optimization algorithm used to update the weights during training (e.g., Adam, SGD, L-BFGS). We selected Adam because it is the good default.
Regularization (α)	Controls the strength of L2 regularization (weight decay) to prevent overfitting by penalizing large weights. We tested different values (e.g., 0.0001, 0.001) and chose 0.0001 to add slight regularization without overly restricting model learning.
Max Iteration	Sets the maximum number of training iterations (epochs) before the algorithm stops, even if convergence isn't reached. We set this to 1000 (default) as a safe upper limit, ensuring sufficient training time while avoiding excessively long runs. Early stopping was also considered to halt training if performance plateaued.
Random Forest	
Number of Trees	Defines the total number of decision trees in the forest. Increasing the number of trees generally improves model performance (by reducing variance) but has diminishing returns and increases computation time.
Number of Attributes at Each Split	Determines how many features are considered for splitting at each node in a tree.
Limit Depth of Individual Trees	Controls the maximum depth of each decision tree in the forest. Deeper trees can model complex patterns but may overfit. We set a maximum depth of 10 to prevent overfitting while allowing sufficient splits. With our small dataset, smaller values (e.g., 5) create simpler trees, trading off some training accuracy for better generalization.
Do Not Split Subsets Smaller Than	[Enabled, Disabled] - Defines the minimum number of samples required to split an internal node. A higher value prevents the model from learning overly specific patterns. With our small dataset, the better choice will be disabled.
Logistic Regression	
Regularization Type	Specifies the type of penalty applied to the model coefficients to prevent overfitting:

	• L1 (Lasso): Shrinks some coefficients to zero, effectively performing feature selection. Useful when dealing with high-dimensional data or when interpretability is important. • L2 (Ridge): Shrinks coefficients evenly without eliminating any, leading to more stable but less sparse solutions. We chose L2 regularization because our dataset does not require explicit feature selection, and L2 generally provides better generalization by distributing penalty across all coefficients.
C Value	Controls the inverse of regularization strength: • Small C (e.g., 0.01, 0.1): Stronger regularization, which helps prevent overfitting but may underfit the data. • Large C (e.g., 1, 10): Weaker regularization, allowing the model to fit the training data more closely but increasing overfitting risk.

3. Information about test and training datasets

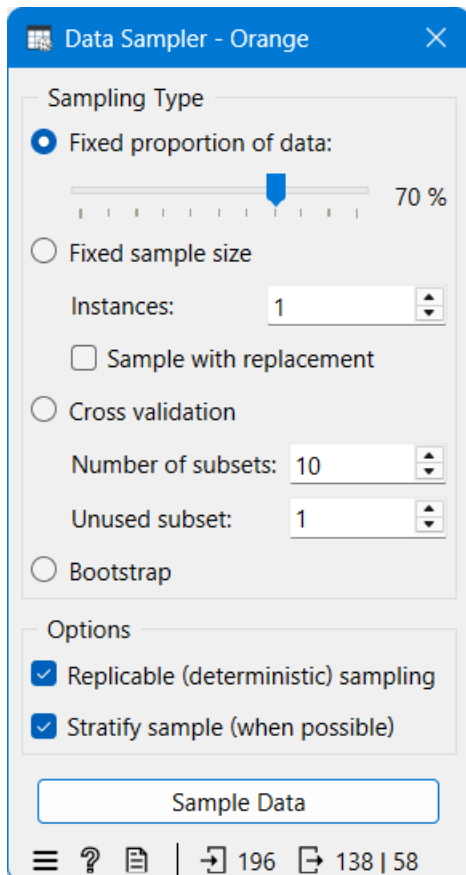


Fig.3.1. Data Sampler in Orange tool

3.1. Number of data objects in the training dataset: 138

3.2. % proportion of data objects in the training dataset:

Class label	Number of data objects in the training dataset	% proportion of data objects in the training dataset
1	49	35.51%
2	49	35.51%
3	12	8.7%
5	6	4.35%
6	5	3.62%
7	17	12.32%
Total	138	100%

3.3. Number of data objects in the test dataset: 58

3.4. % proportion of data objects in the test dataset:

Class label	Number of data objects in the test dataset	% proportion of data objects in the test dataset
1	21	36.21%
2	20	34.48%
3	5	8.62%

5	3	5.17%
6	2	3.45%
7	7	12.07%
Total	58	100%

4. Experiments with artificial neural network

Experiment	Hyperparameter values
Experiment 1	Neurons in hidden layer: 20, 15 Activation: Logistic, Solver: L-BFGS-B Regularization: 0.0005 Maximal number of iterations: 1000
Experiment 2	Neurons in hidden layer: 25 Activation: tanh, Solver: L-BFGS-B Regularization: 0.001 Maximal number of iterations: 500
Experiment 3	Neurons in hidden layer: 20, 10, 5 Activation: Identity, Solver: Adam Regularization: 0.005 Maximal number of iterations: 10000
Experiment 4	Neurons in hidden layer: 25 Activation: ReLu, Solver: Adam Regularization: 0.0001 Maximal number of iterations: 1000

Hyperparameter values for Experiment 1:

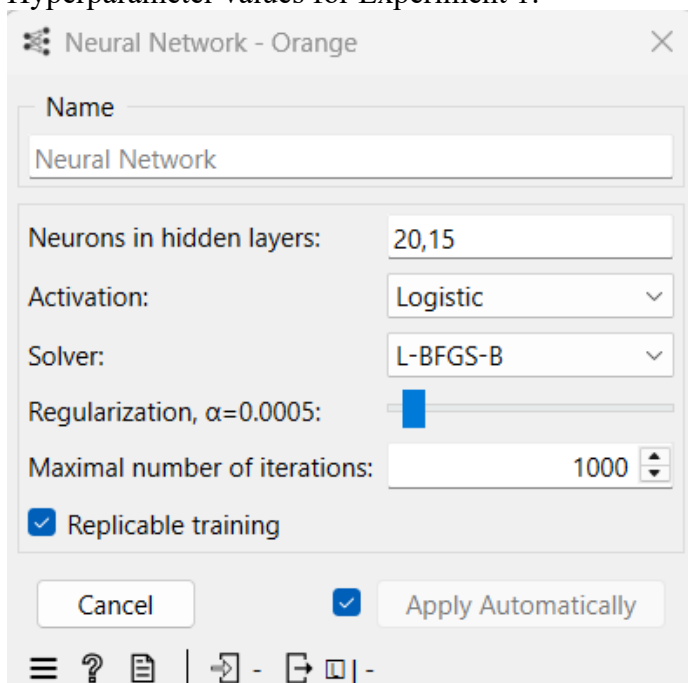


Fig.3.2 Neural network parameters for experiment 1

Performance metrics for Experiment 1:

Cross Validation:

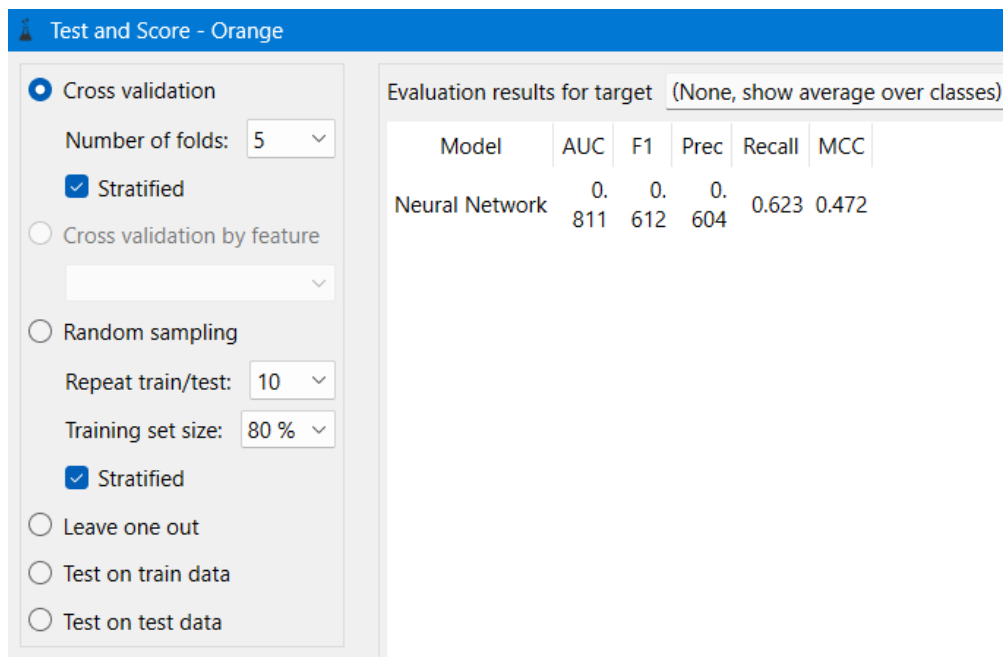


Fig.3.3 Neural network cross validation for experiment 1

Precision is quite good but tends to be lower than Recall in CV -> data is noisy/mixed between classes. In this model, Precision is not a weakness, but it is not too outstanding in CV (cross-validation). Since Recall and Precision are close together, F1 is average to good.

Experiment 2:

Hyperparameter values for Experiment 2:

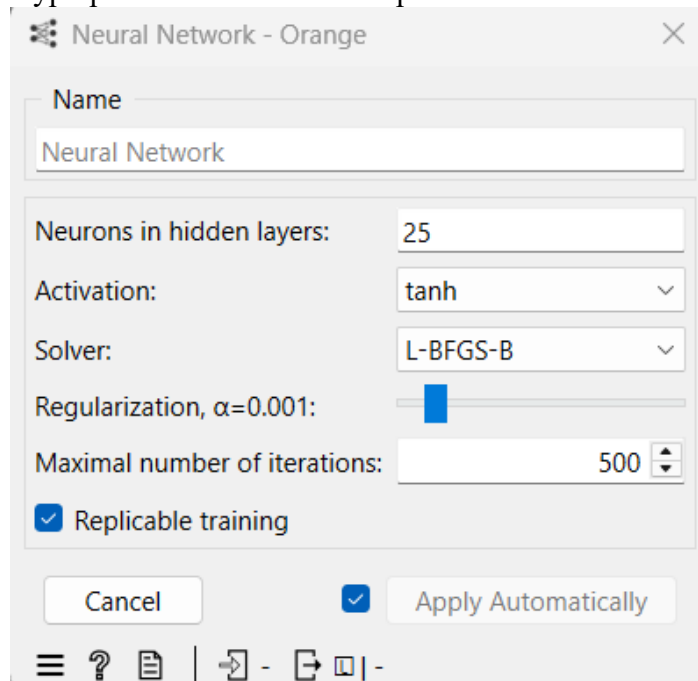


Fig.3.4 Neural network parameters for experiment 2

Performance metrics for Experiment 2:

Cross Validation:

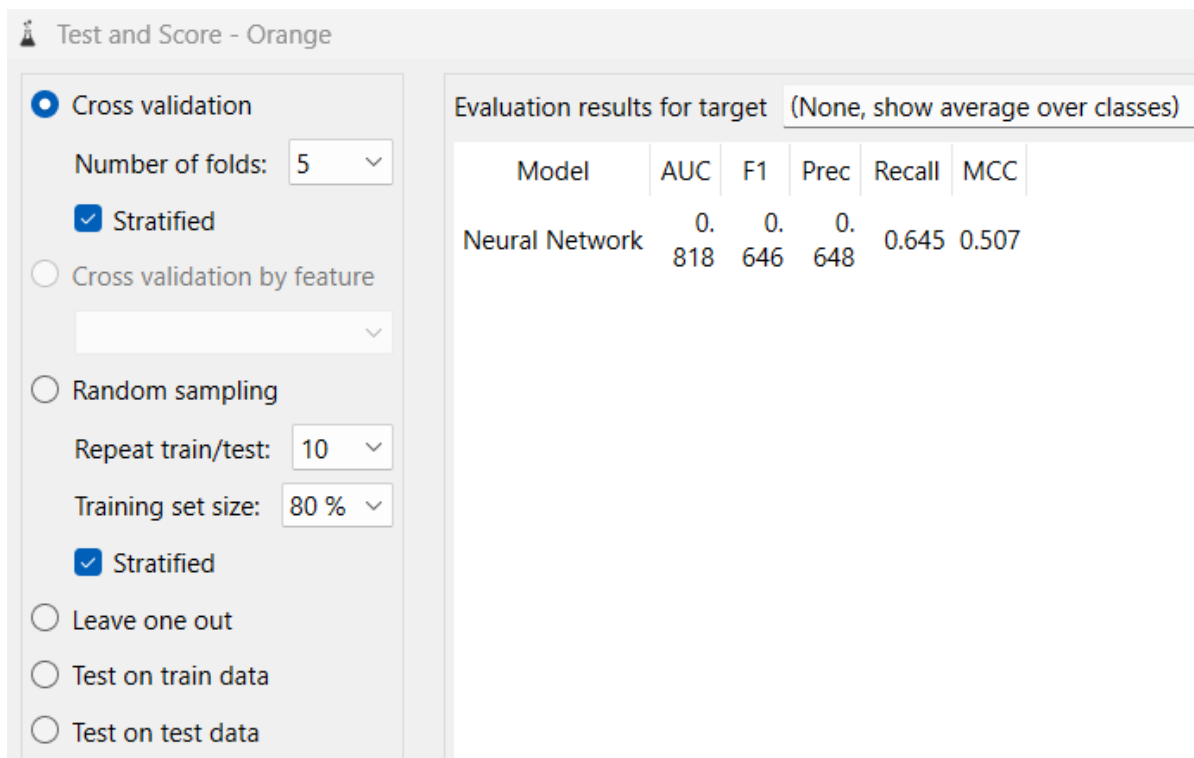


Fig.3.5 Neural Network cross validation for experiment 2

Recall and Precision are close together, F1 is average to good.

Hyperparameter values for Experiment 3:

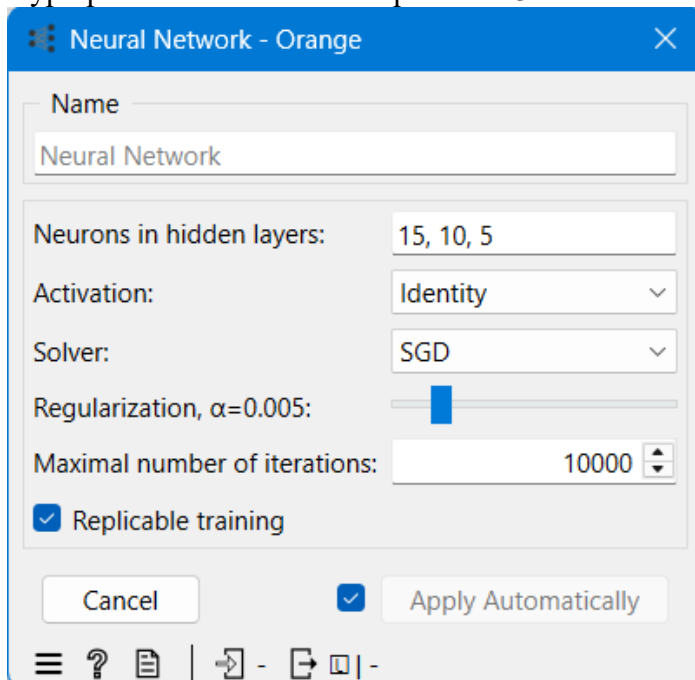


Fig.3.6 Neural network parameters for experiment 3

Performance metrics for Experiment 3:

Cross Validation:

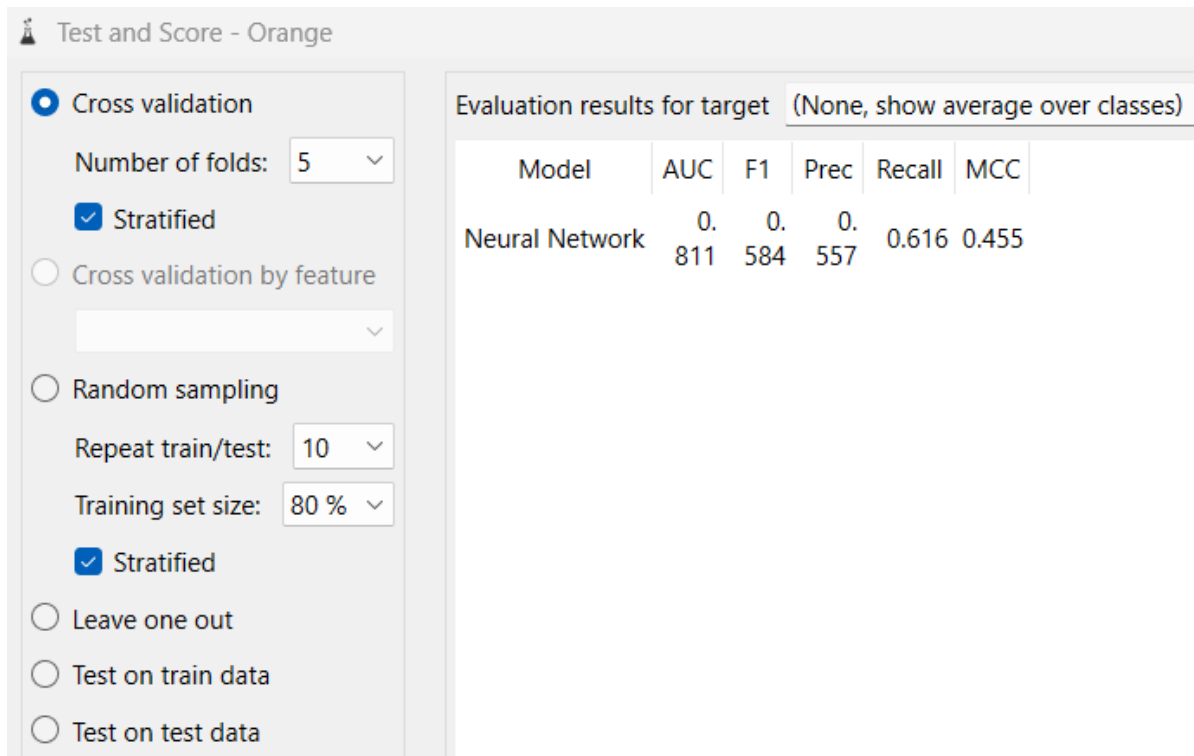


Fig.3.7 Neural Network cross validation for experiment 3

Precision in CV is the lowest in 4 experiments – due to the parameters SGD and Identity. Recall is higher than Precision in CV, showing that the model tends to cover more → limit missing correct samples. F1-score is quite average in CV. F1 in CV is still lower than previous models using tanh or logistic → showing that activation identity may not be optimal in hidden layers for this problem.

Hyperparameter values for Experiment 4:

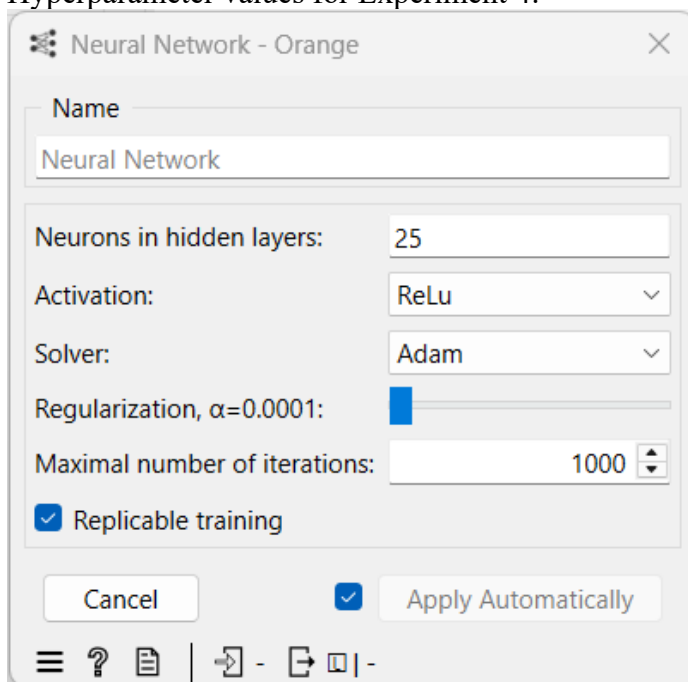


Fig.3.8 Neural Network parameters for experiment 4

Performance metrics for Experiment 4:
Cross Validation:

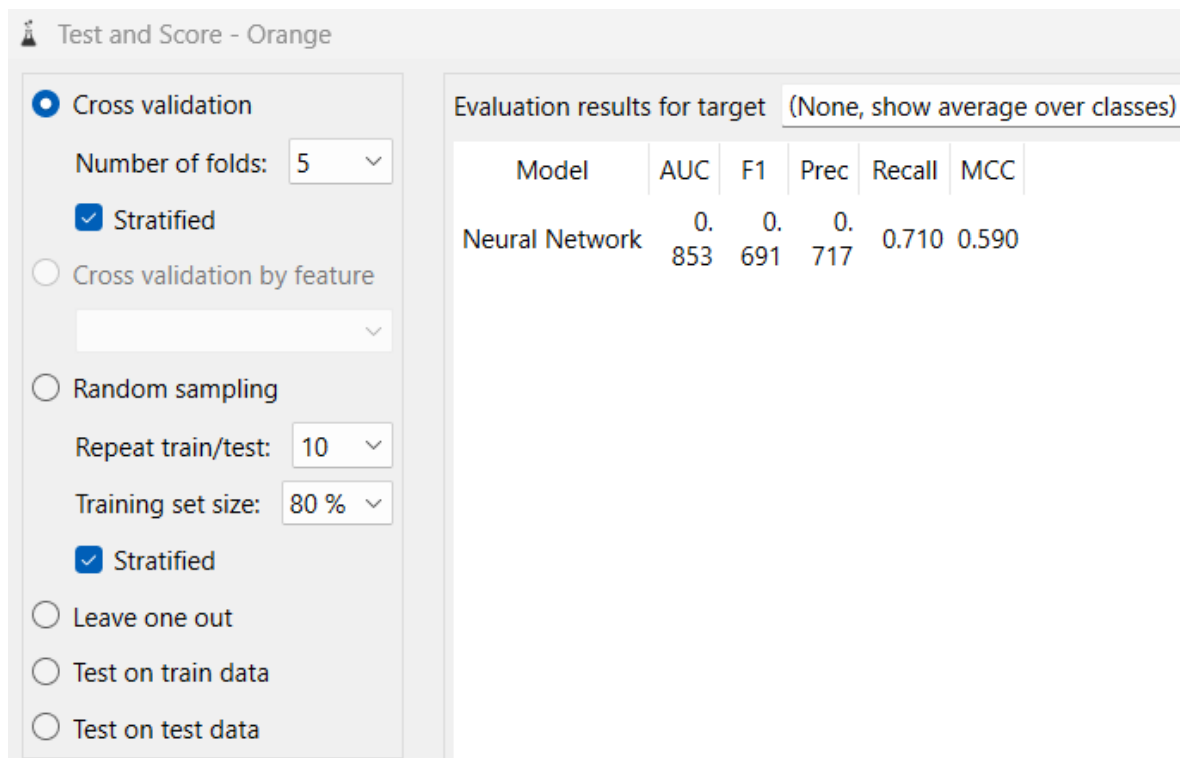


Fig.3.9 Neural Network cross validation for experiment 4

Precision is very high → showing that the model has very few false positives. ReLU combined with Adam helps the model learn well. Recall is also very high → the model maintains excellent coverage. The highest F1-score of all the models that we have tried.

Conclusions from experiments:

Activation function (ReLU and Logistic) give the best results, with significantly higher F1 and AUC than Identity. Tanh gives quite good results, but not outstanding. Identity is not suitable for hidden layers in this problem.

Solver - Adam (used in ReLU model) helps the model learn fast, stable and generalize well → suitable for moderate data. L-BFGS-B works well with Logistic and Tanh, giving stable results. SGD gives the worst performance → not recommended in this case.

The number of neurons - The 1 hidden layer model (25 neurons) gives the best performance (ReLU & Tanh). The 2 hidden layer model (20, 15) also performs well. The 3 layer model (15,10,5) does not perform better → showing that increasing depth is not always effective, especially with small data.

F1, Precision and Recall all increased slightly in the test set → the model has good generalization ability, not overfitting.

ReLU + Adam (25 neurons) – Best Performing Model

Logistic + L-BFGS-B (20,15 neurons) – Close Runner-Up

Tanh + L-BFGS-B (25 neurons) – Moderately Strong

Identity + SGD (15,10,5 neurons) – Weakest Performer

4.1. Model selected for testing:

Experiment 4, with parameters are shown like *Fig.3.13 Neural network for experiment 4* is the best.

- Neurons in hidden layers: 25
- Activation function: ReLU
- Solver: Adam
- Regularization parameter (α): 0.0001
- Maximal number of iterations: 1000

It achieved the highest F1-score on the test set, indicating a strong balance between Precision and Recall.

It recorded the highest Precision among all tested models, meaning it was the most accurate in its predictions. The Recall was also consistently high, demonstrating its ability to correctly identify the true class labels without omitting too many instances. The AUC and MCC values further confirmed the model's reliability and robustness across multiple classes, even in the presence of class imbalance in the dataset.

5. Experiments with Random Forest

Experiment	Hyperparameter values
Experiment 1	Number of trees: 6 Number of attributes: considered at each split: 11 Limit depth of individual trees: 3 Do not split subsets smaller than: (Disabled)
Experiment 2	Number of trees: 100 Number of attributes: considered at each split: 3 Limit depth of individual trees: 6 Do not split subsets smaller than: (Disabled)
Experiment 3	Number of trees: 1500 Number of attributes: considered at each split: 3 Limit depth of individual trees: 6 Do not split subsets smaller than: 5 (Enabled)
Experiment 4	Number of trees: 50 Number of attributes: considered at each split: 6 Limit depth of individual trees: 3 Do not split subsets smaller than: 5 (Enabled)

Hyperparameter values for Experiment 1:

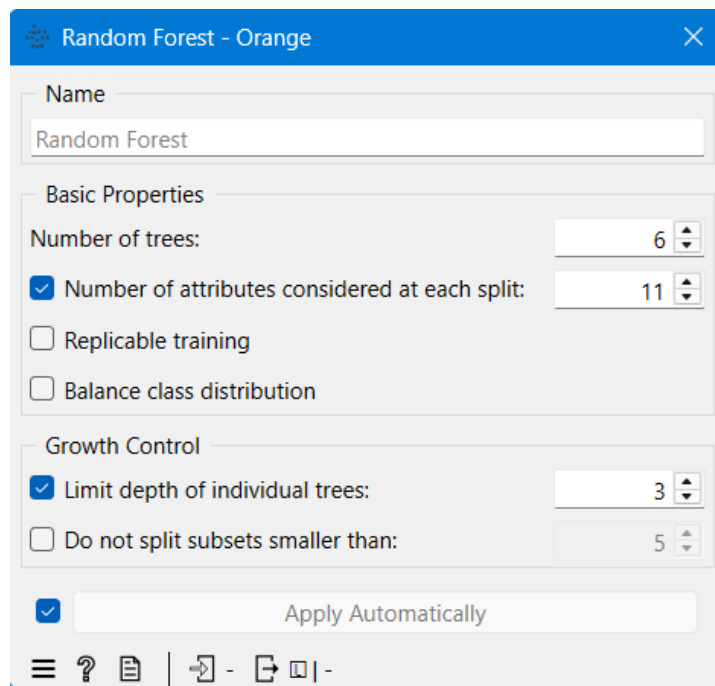


Fig.3.10 Parameter in Experiment 1

Performance metrics for Experiment 1:

Cross Validation:

Evaluation results for target (None, show average over classes)					
Model	AUC	F1	Prec	Recall	MCC
Random Forest	0.828	0.656	0.655	0.696	0.567

Fig.3.11 Cross validation in Experiment 1

Hyperparameter values for Experiment 2:

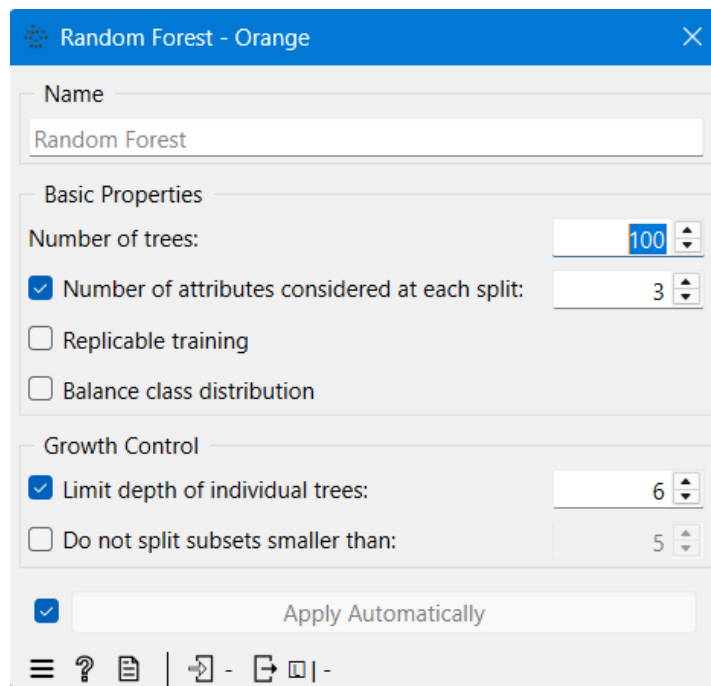


Fig.3.12 Parameter in Experiment 2

Performance metrics for Experiment 2:

Cross Validation:

Test and Score - Orange

☒ Cross validation

Number of folds: 5

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 80 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☐ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	F1	Prec	Recall	MCC
Random Forest	0.878	0.710	0.766	0.732	0.625

Fig.3.13 Cross validation in Experiment 2

Hyperparameter values for Experiment 3:

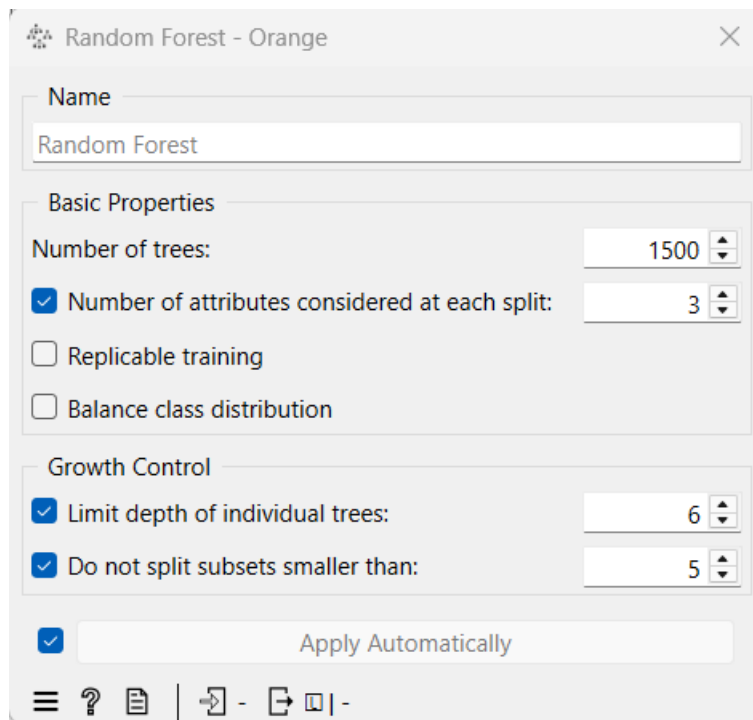


Fig.3.14 Parameter in Experiment 3

Performance metrics for Experiment 3:

Cross Validation:

Test and Score - Orange

☒ Cross validation

Number of folds: 5

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 80 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☐ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	F1	Prec	Recall	MCC
Random Forest	0.876	0.703	0.758	0.725	0.613

Fig.3.15 Cross validation in Experiment 3

Hyperparameter values for Experiment 4:

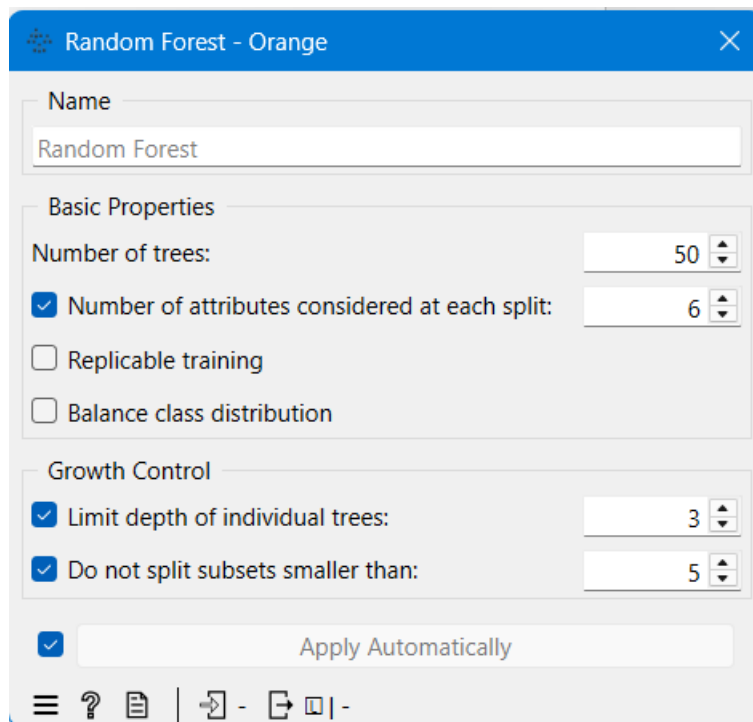


Fig.3.16 Parameter in Experiment 4

Performance metrics for Experiment 4:

Cross Validation:

Model	AUC	F1	Prec	Recall	MCC
Random Forest	0.848	0.672	0.654	0.710	0.594

Fig.3.17 Cross validation in Experiment 4

Conclusions from experiments:

The experiments involved training several random forest models with different hyperparameter settings and evaluating their performance using 5-fold cross-validation with stratification. The key metrics observed are AUC (Area Under the Curve), F1 score, Precision, Recall, and MCC (Matthews Correlation Coefficient)

- Experiment 1: With 6 trees, 11 attributes, and a tree depth of 3, the model achieved an AUC of 0.828, F1 score of 0.656, Precision of 0.655, Recall of 0.696, and MCC of 0.567.
- Experiment 2: With 100 trees, 3 attributes, and a tree depth of 6, the model showed improved performance with an AUC of 0.878, F1 score of 0.710, Precision of 0.766, Recall of 0.732, and MCC of 0.625.
- Experiment 3: Increasing the number of trees to 1500 while keeping other parameters (3 attributes, depth of 6) nearly the same resulted in an AUC of 0.876, F1 score of 0.703, Precision of 0.758, Recall of 0.725, and MCC of 0.613. This indicates a slight decrease in performance compared to Model 2.
- Experiment 4: A model with 50 trees, 6 attributes, and a tree depth of 3, yielded an AUC of 0.848, F1 score of 0.672, Precision of 0.654, Recall of 0.710, and MCC of 0.594.

5.1. Model selected for testing:

Based on the evaluation results, Experiment 2, which has 100 trees, 3 attributes, and a tree depth of 6, appears to be the most promising model for testing, because it has the highest values for AUC, F1 score, Precision, Recall, and MCC. This suggests a good balance between precision and recall, which is essential for a reliable classifier.

6. Experiments with Logistic Regression

Experiment	Hyperparameter values
Experiment 1	Regularization: L2 (Ridge) C = 550
Experiment 2	Regularization: L2 (Ridge) C = 0.080
Experiment 3	Regularization: L1 (Lasso) C = 1.0
Experiment 4	Regularization: L1 (Lasso) C = 200

Hyperparameter values for Experiment 1:

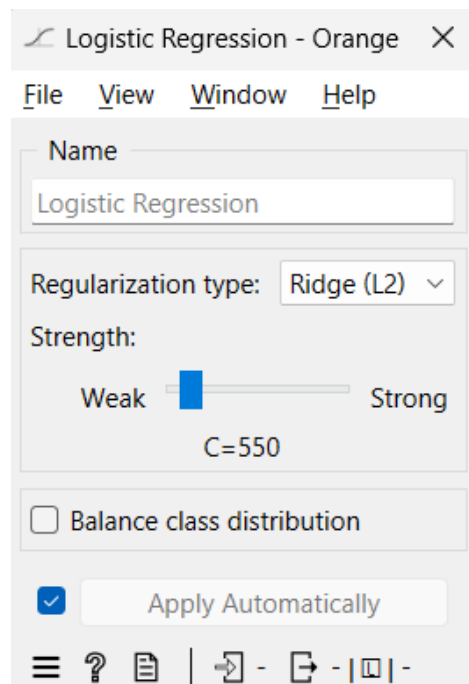


Fig.3.18 Parameter in Experiment 1

Performance metrics for Experiment 1:

Cross Validation:

Test and Score - Orange

☒ Cross validation

Number of folds: 5

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 80 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☐ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	F1	Prec	Recall	MCC
Logistic Regression	0.817	0.664	0.638	0.696	0.571

Fig.3.19 Cross validation in Experiment 1

Hyperparameter values for Experiment 2:

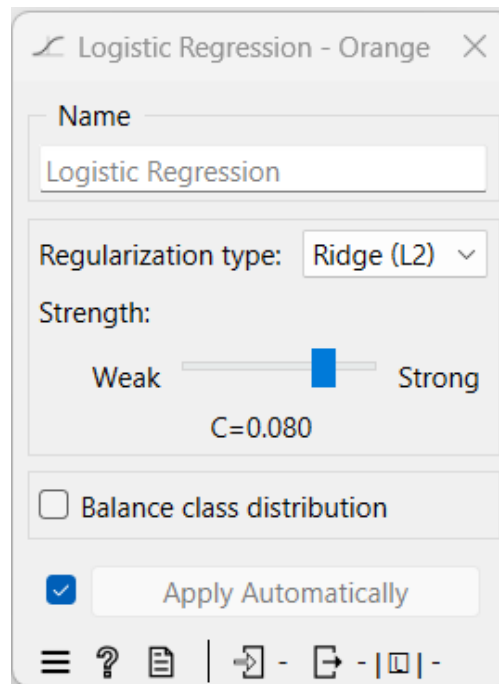


Fig.3.20 Parameter in Experiment 2

Performance metrics for Experiment 2:

Cross Validation:

Test and Score - Orange

☐ Cross validation
 Number of folds: 5
☒ Stratified

☐ Cross validation by feature

☐ Random sampling
 Repeat train/test: 10
 Training set size: 80 %
☒ Stratified

☐ Leave one out
☐ Test on train data
☒ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	F1	Prec	Recall	MCC
Logistic Regression	0.769	0.412	0.351	0.500	0.243

Fig.3.21 Cross validation in Experiment 2

Hyperparameter values for Experiment 3:

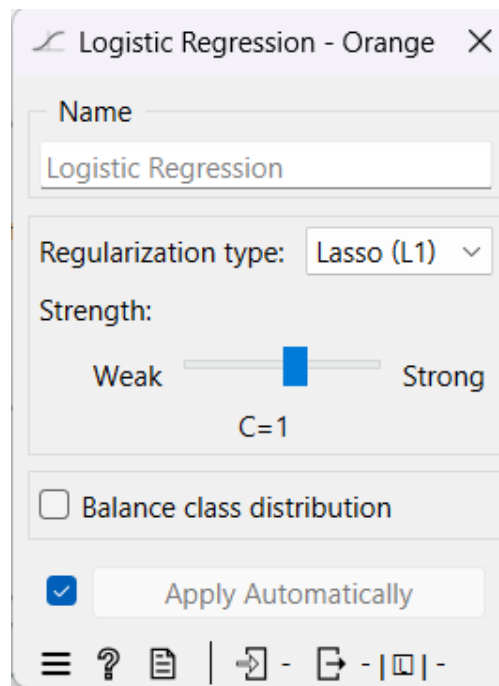


Fig.3.22 Parameter in Experiment 3

Performance metrics for Experiment 3:

Cross Validation:

Test and Score - Orange

☐ Cross validation

Number of folds: 5

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 80 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☒ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	F1	Prec	Recall	MCC
Logistic Regression	0.778	0.480	0.439	0.534	0.325

Fig.3.23 Cross validation in Experiment 3

Hyperparameter values for Experiment 4:

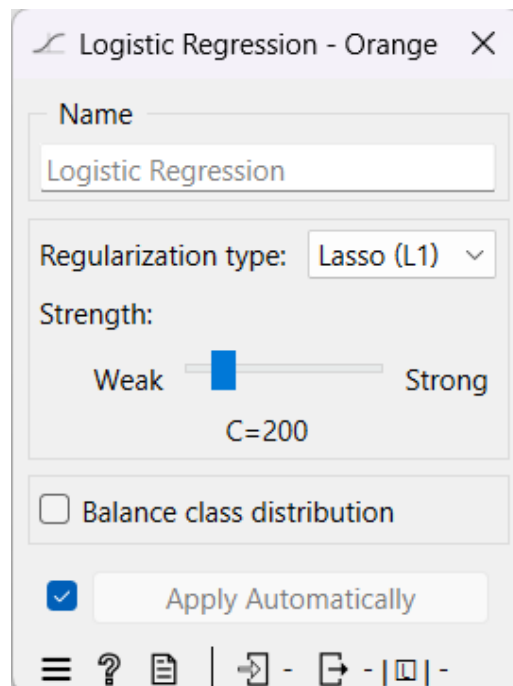


Fig.3.24 Parameter in Experiment 4

Performance metrics for Experiment 4:
Cross Validation:

Test and Score - Orange

☐ Cross validation

Number of folds: 5

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 80 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☒ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	F1	Prec	Recall	MCC
Logistic Regression	0.832	0.641	0.616	0.672	0.544

Fig.3.25 Cross validation in Experiment 4

Conclusions from experiments:

1. First Experiment (Ridge L2, C=550)
 - Metrics: AUC = 0.817, F1 = 0.664, Precision = 0.638, Recall = 0.696, MCC = 0.571
 - Analysis: High regularization strength (C=550) led to the best overall performance, with strong AUC and balanced precision/recall. The model generalizes well, as indicated by the high MCC (Matthews Correlation Coefficient), which accounts for class imbalance.
2. Second Experiment (Ridge L2, C=0.080)
 - Metrics: AUC = 0.769, F1 = 0.412, Precision = 0.351, Recall = 0.500, MCC = 0.243
 - Analysis: Weak regularization (C=0.080) caused significant performance drops, especially in precision and F1. The model likely underfits, struggling to capture patterns due to excessive penalty on coefficients.
3. Third Experiment (Lasso L1, C=1)
 - Metrics: AUC = 0.778, F1 = 0.480, Precision = 0.439, Recall = 0.534, MCC = 0.325
 - Analysis: L1 regularization (sparse feature selection) improved results compared to weak Ridge but still underperformed compared to the first experiment. The trade-off between precision and recall is less optimal.
4. Fourth Experiment (Lasso L1, C=200, no class balancing)
 - Metrics: AUC = 0.832, F1 = 0.641, Precision = 0.616, Recall = 0.672, MCC = 0.544
 - Analysis: Stronger L1 regularization (C=200) yielded the highest AUC, suggesting better class separation. However, F1 and MCC are slightly lower than the first experiment, indicating Ridge (L2) may handle class imbalance better without explicit balancing.

6.1. Model selected for testing:

Based on the comparative performance of five Logistic Regression configurations, the model selected for final testing is from Experiment 1, which utilized:

- Regularization Type: L2 (Ridge)
- C Value: 550
- Class Balancing: Disabled
- Cross-Validation: 5-fold stratified

This model consistently delivered the strongest performance across all key metrics:

- F1 Score: 0.664
- Recall: 0.696
- MCC: 0.571
- AUC: 0.817

Despite the absence of class balancing, the model achieved excellent recall and maintained a high precision-recall balance, suggesting it handled class distribution effectively on its own. The combination of minimal regularization and L2 penalty preserved predictive capacity while avoiding overfitting.

As a result, this model is considered the most robust and generalizable, and will be used for evaluation on the test dataset.

Testing results of the trained models

Best outcomes from Artificial Neural Network:

Cross Validation:

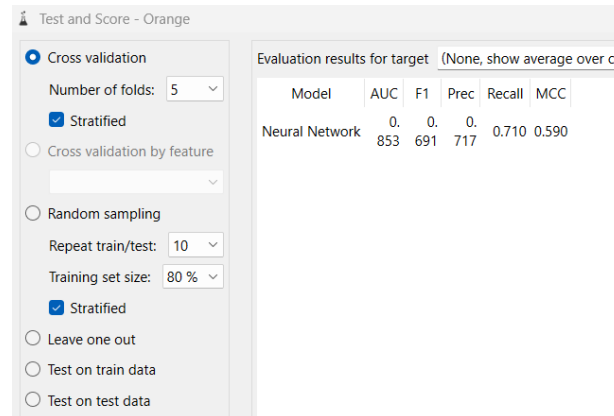


Fig.3.26 Cross validation for experiment 4

Test on test data:

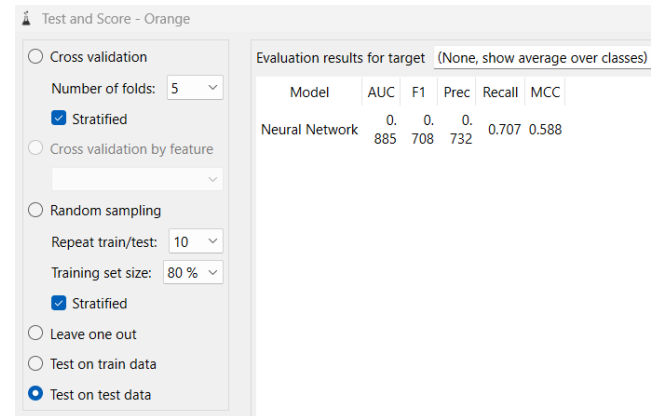


Fig.3.27 Test on test data for experiment 4

Precision
CV: 0.717
Test: 0.732

Recall
CV: 0.710
Test: 0.707

F1-score
CV: 0.691
Test: 0.708

Precision is very high and stable in both CV and test → showing that the model has very few false positives.

Recall is also very high and almost unchanged between CV and test → the model maintains excellent coverage.

The highest F1-score of all the models that we have tried. The stability between CV and test shows that the model has extremely good generalization ability, without overfitting.

Best outcomes from Random Forest:

Cross Validation:

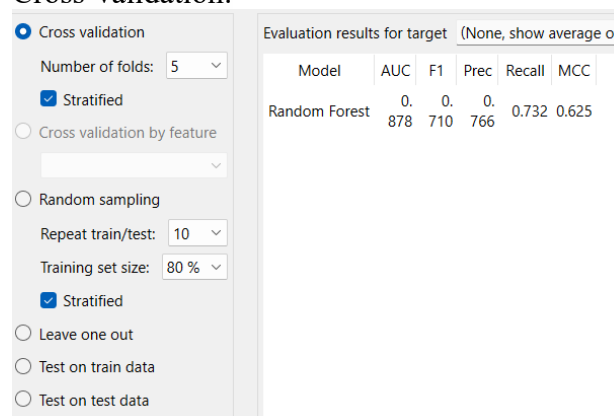


Fig.3.28 Cross validation for experiment 2

Test on test data:

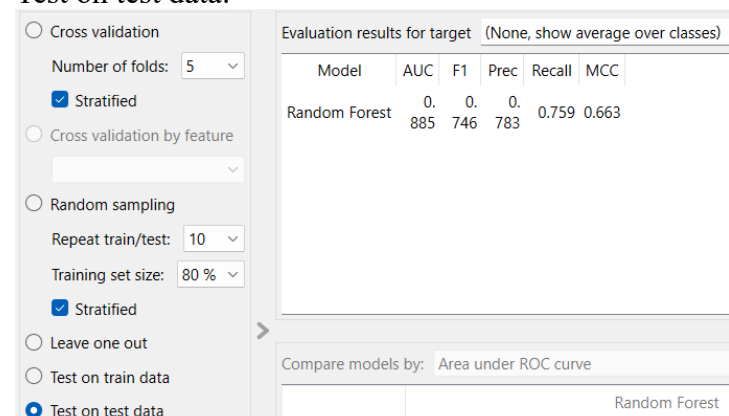


Fig.3.29 Test on Test data – Experiment 2

Precision
CV: 0.766
Test: 0.783

Recall
CV: 0.732
Test: 0.759

F1-score
CV: 0.710
Test: 0.746

Precision - For both models in CV and Test data have Few false positives (high confidence in positive predictions).

- The model achieves high precision (0.783), meaning it rarely makes false positive errors.
- It also has high recall (0.759), meaning it captures most true positives.
- The F1-score (0.746) shows an excellent balance between these two metrics.

Performance slightly improves on the test set (vs. cross-validation), proving the model works well on new, unseen data.

Best outcomes from Logistic Regression:

Cross Validation:

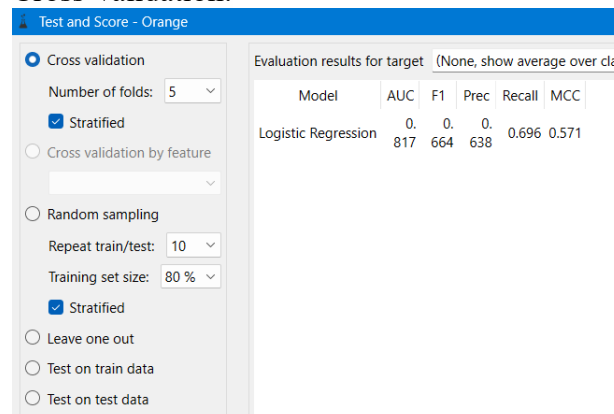


Fig.3.30 Cross validation for experiment 1

Test on test data:

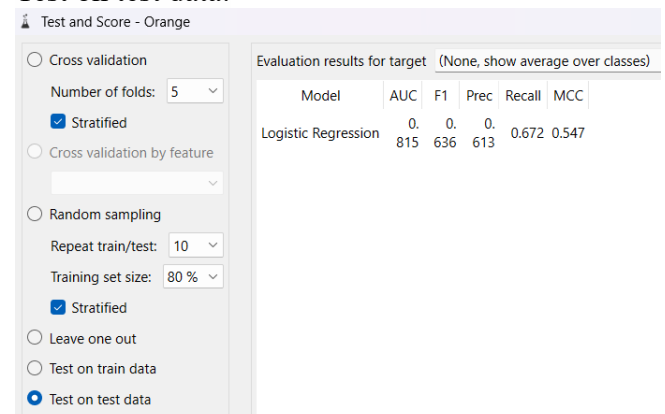


Fig.3.31 Test on Test data for experiment 1

Precision
CV: 0.638
Test: 0.613

Recall
CV: 0.696
Test: 0.672

F1-score
CV: 0.664
Test: 0.636

- Precision shows moderate performance with a slight drop in the test set (0.638 → 0.613), indicating some false positives in predictions.
- Recall demonstrates the model's best capability (highest metric) but also experiences a small decrease from CV to test (0.696 → 0.672), suggesting minor sensitivity to unseen data.
- F1-score maintains reasonable balance but follows the downward trend (0.664 → 0.636), reflecting the combined effect of precision and recall changes.

Overall:

- All metrics show <5% relative difference between CV and test results, indicating stable generalization.

- The pattern suggests slight underfitting rather than overfitting, as performance doesn't degrade significantly on unseen data.

Best Performing Models (Test on Test Data)

Algorithm	Precision	Recall
Logistic Regression	0.638	0.696
Random Forest	0.783	0.759
Neural Network	0.732	0.707

Comparison and Best Algorithm Selection

1. Precision (Higher = Fewer False Positives)
 - Random Forest (0.783) > Neural Network (0.732) > Logistic Regression (0.638)
 - Logistic Regression has the lowest precision, indicating it produces the most false positives among the three models. Random Forest remains the most precise classifier.
2. Recall (Higher = Fewer False Negatives)
 - Random Forest (0.759) > Neural Network (0.707) > Logistic Regression (0.696)
 - Random Forest captures the truest positives, making it the strongest model in terms of sensitivity.

Best Overall Algorithm: Random Forest - It achieves the highest recall (0.759) while maintaining the highest precision (0.783).

Neural Network: Balanced but Outperformed by Random Forest: The Neural Network offered a solid balance with precision (0.732) and recall (0.707), but it was outperformed by Random Forest in both metrics.

Conclusion:

If precision is the only concern, Random Forest (0.783) remains the best choice. Logistic Regression may still be valuable where model simplicity is critical. However, Random Forest offers the best overall trade-off between minimizing false positives and maximizing true positives, making it the top recommendation for most real-world use cases.

Precision Performance Comparison

Algorithm	Precision	Recall	Key Observation
Logistic Regression	0.638	0.696	- Lowest precision (63.8%) → Prone to false positives (incorrectly classifying some glasses). - Moderate recall (69.6%) → Misses approx.30% of actual positives. - Best for simplicity but not ideal for high-stakes decisions where errors are costly.
Random Forest	0.783	0.759	- Best precision (78.3%) → Fewest false positives. - Best recall (75.9%) → Captures most true positives. - Ideal for forensic applications where both accuracy and reliability matter.
Neural Network	0.732	0.707	- Balanced precision (73.2%) and recall (70.7%). - Better than Logistic Regression but outperformed by Random Forest. - Good alternative if interpretability is less critical.

Observation:

- Random Forest demonstrates the strongest precision (0.783), making it ideal for applications where false positives are particularly costly (e.g., forensic evidence analysis or medical diagnostics), while also offering the highest recall.
- While Neural Network shows slightly lower precision (0.732), it maintains a good balance across metrics, making it suitable when moderate performance is acceptable in both precision and recall.
- Logistic Regression trails in precision performance (0.638) and does not offer compensatory advantages in this metric-focused evaluation. It may still be useful in contexts that prioritize model simplicity and explainability.

Information sources

1. Hierarchical Clustering

- **Video Tutorials:**
 - [Hierarchical Clustering \(Agglomerative\) - Step-by-Step Explanation](#)
 - [Hierarchical Clustering Explained Simply](#)
 - [Hierarchical Clustering: Single, Complete & Average Linkage](#)
- **Reference:**
 - [Module 4.3: Unsupervised Learning & Types of Linkages in Hierarchical Clustering](#)
 - [Unsupervised Learning and Types of Linkages in Hierarchical Clustering - GeeksforGeeks](#)

Hyperparameter – Linkage Method	Description
Single Linkage	Single Linkage determines cluster similarity based on the shortest distance between any two points from the two clusters — that is, it considers the distance between the closest pair of elements in each cluster. Given the characteristics of single linkage — particularly its sensitivity to outliers and its tendency to merge clusters based on just a few closely positioned points — it may not be ideal for our case. Although we already applied Isolation Forest in the Orange tool to filter out outliers, our dataset is relatively small and initially contained many outliers. To minimize any remaining impact and ensure more stable clustering, we decided to use average linkage instead.
Average Linkage	Average Linkage defines the distance between two clusters as the average of the distances between all possible pairs of points. Among the various types of linkage methods in hierarchical clustering, average linkage strikes a balance between single and complete linkage by considering all point-to-point distances, rather than focusing solely on the nearest or farthest points. This typically produces clusters that are moderately compact.
Complete Linkage	For two clusters, complete linkage calculates the distance as the maximum distance between any pair of points, one from each cluster. This method tends to produce tight and spherical clusters, as it is sensitive to outliers and aims to prevent clusters from being too widely spread. However, given the characteristics of our dataset, it wouldn't be an appropriate choice - the same reason with the single linkage.
Ward's Linkage	Ward's Linkage measures the distance between two clusters based on the increase in total variance (or error sum of squares) that would result from merging them. This method aims to form compact and well-separated clusters by minimizing the variability within each cluster, ensuring that the data points in a cluster remain as similar to one another as possible. It is the most suitable method with our dataset.

2. Random Forest

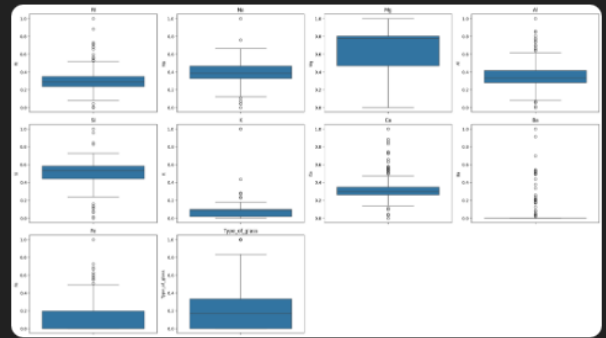
- **Articles & Guides:**
 - [Random Forest Algorithm in Machine Learning \(GeeksforGeeks\)](#)
 - [IBM Guide: What is Random Forest?](#)

3. Logistic Regression

- **Beginner-Friendly Guides:**
 - [Multinomial Logistic Regression for Beginners](#) (*Kaggle*)
 - [Understanding Logistic Regression](#) (*GeeksforGeeks*)
- **Video Explanations:**
 - [Logistic Regression Explained](#)
 - [Logistic Regression for Classification](#)

Appendix

1. Analyzing Outliers with Boxplots:



Nhận xét outlier giúp mình nhaaa

✓ Features with Significant Outliers:

1. RI (Refractive Index):

- Multiple values lie far above and below the interquartile range.
- This indicates possible anomalies in glass density or composition.

2. Na (Sodium):

- Exhibits both high and low outliers, suggesting variability in sodium content across different glass types.
- These could influence the chemical stability or melting point of the glass.

3. Al (Aluminum):

- Numerous high-end outliers suggest certain glass samples are aluminum-enriched.
- These may correspond to specialized glass types with improved durability.

4. Si (Silicon):

- Outliers exist at both extremes, especially at the lower end.
- Since silicon is the primary component in glass, any deviation could be chemically significant.

5. Ca (Calcium):

- A large number of both low and high outliers indicate variation in stabilizer content.
- These differences could affect the hardness and chemical resistance of glass.

6. Ba (Barium):

- Exhibits the highest number of outliers.
- Many samples have values close to 0, while a few have significantly high barium content.
- This indicates a skewed distribution, likely due to certain glass types incorporating barium for optical purposes.

7. Fe (Iron):

- High outliers dominate the upper tail.
- Iron may have been added intentionally for coloring or UV-blocking, or could indicate impurities.

● Features with Mild or Few Outliers:

1. Mg (Magnesium):

- Distribution is relatively even.
- No extreme values observed, suggesting consistent use across glass types.

2. K (Potassium):

- Few high outliers present.
- These may relate to specific glass treatments ↓ formulations (e.g., toughened glass).

2. At the beginning of hierarchical clustering, is it better to analyze data with or without outliers?

hierarchical clustering do we need to remove the outliers before training hierarchical clustering and why

In hierarchical clustering, removing outliers before training is generally recommended, but not strictly required. Here's why:

✓ Why You Should Remove Outliers Before Hierarchical Clustering:

1. Distortion of Dendrogram Structure:

Outliers can form their own separate clusters or artificially increase the distance between clusters, distorting the dendrogram (tree diagram) and making it harder to interpret true structure.

2. Affecting Linkage Methods:

Most linkage methods (e.g., single, complete, average) rely on distance calculations. Since outliers tend to be far from the main data groups, they can inflate distances, leading to incorrect merges and splits.

3. Reduced Cluster Quality:

Including outliers may result in clusters that are not compact or meaningful, reducing the usefulness of the final grouping.

⚠ When You Might Not Remove Them:

- If detecting outliers is your goal, hierarchical clustering (especially with distance threshold cuts) can be used as a tool for outlier detection.
- In some small datasets, keeping outliers might not severely impact results — but it depends on the data distribution.

🧠 Best Practice:

- Visualize the data (e.g., with PCA or t-SNE) to identify outliers.
- Use statistical methods (e.g., z-score, IQR) or DBSCAN before hierarchical clustering to filter them.
- Then, apply hierarchical clustering to the cleaned dataset for more reliable cluster structures.

Would you like a quick example of how outliers affect a hierarchical dendrogram?

and kmeans before seeing the silhouette score after training what we should do? Do we have to remove the outliers or not? Why we should remove and why not?

In K-means clustering, it's generally advisable to **remove outliers before training**, especially when evaluating cluster quality using metrics like the **silhouette score**. Here's why:

✓ Why Remove Outliers Before K-Means Clustering

1. K-Means is Sensitive to Outliers

K-means minimizes the sum of squared distances between data points and their assigned cluster centroids. Outliers, being far from other data points, can disproportionately influence these centroids, leading to distorted cluster formations.

2. Improved Silhouette Scores

The silhouette score measures how similar a data point is to its own cluster compared to other clusters. Outliers often have low or negative silhouette scores, indicating poor clustering. Removing them can lead to higher average silhouette scores, reflecting better-defined clusters.

3. Enhanced Cluster Interpretability

Outliers can form their own clusters or skew existing ones, making it challenging to interpret the

3. Enhanced Cluster Interpretability

Outliers can form their own clusters or skew existing ones, making it challenging to interpret the clustering results meaningfully. By removing outliers, the resulting clusters are more representative of the underlying data structure.

⚠ When You Might Not Remove Outliers

- **Outlier Detection as a Goal**

If identifying outliers is part of your analysis objective, retaining them is essential. In such cases, clustering can help highlight these anomalies.

- **Robust Clustering Algorithms**

Alternative clustering methods like **DBSCAN** are more robust to outliers and can handle noise effectively without the need for prior outlier removal. Cross Validated

🧠 Best Practices for Handling Outliers in K-Means

1. Preprocessing Steps

- **Outlier Detection:** Use statistical methods (e.g., z-score, IQR) or clustering-based techniques to identify outliers.



Best Practices for Handling Outliers in K-Means

1. Preprocessing Steps

- **Outlier Detection:** Use statistical methods (e.g., z-score, IQR) or clustering-based techniques to identify outliers.
- **Outlier Removal:** Consider removing or treating identified outliers before applying K-means.

2. Evaluate Clustering Quality

- **Silhouette Analysis:** Calculate silhouette scores to assess the quality of clustering and the impact of outliers.
- **Compare Results:** Perform clustering with and without outliers to understand their effect on the clustering outcome.

3. Alternative Clustering Methods

If outliers significantly impact K-means results, explore other clustering algorithms like **DBSCAN** or **hierarchical clustering**, which may be more suitable depending on the data characteristics.

In summary, while it's not mandatory to remove outliers before applying K-means clustering, doing so can lead to more meaningful and interpretable clusters, especially when evaluating cluster quality using silhouette scores.

3. About the parameter in Orange tools:

Regularization has 3 options in orange là Lasso L1 Ridge L2 và None
It has C - what is that - if bigger value what it will happen - if not?
And Balance class distribution - what is that - is it different between
enable and disable them?

Can you tell them in English

Absolutely! Here's a clear and beginner-friendly explanation in **English** of the three hyperparameters in Orange's Logistic Regression widget:

⚙️ 1. Regularization Type

Regularization helps prevent the model from **overfitting** by adding a penalty to the model's complexity.

In **Orange**, there are three options:

- **Lasso (L1)**
 - Adds a penalty based on the **absolute values** of the coefficients.
 - It can **shrink some coefficients to zero**, which helps with **feature selection** (removing unimportant features).
↓
- **None**
 - No regularization is applied.
 - The model may **overfit** (memorize training data too much), especially if there are many features or noise in the data.

⚙️ 2. C (Inverse of Regularization Strength)

- **C** controls how strong the regularization is.
 - **Small C** → Strong regularization (model is simpler, less likely to overfit).
 - **Large C** → Weak regularization (model fits the training data more, may overfit).

So:

- If you set **C = 1000**, the regularization is very **weak** → model may overfit.
- If you set **C = 0.01**, the regularization is **strong** → model becomes simpler and more general.

Rule of thumb: Try **C = 1** first, then adjust based on performance.

⚙️ 3. Balance Class Distribution

- This setting is useful when your **target classes are imbalanced**, meaning some classes have many more samples than others.
- **When enabled:**
 - The model automatically gives more weight to the minority (smaller) classes so they are not ignored during training.
- **When disabled:**
 - All classes are treated equally. If some classes have very few examples, the model may **perform poorly** on them.

✅ Summary:

Setting	Effect
Regularization Type	L1 (removes features), L2 (shrinks but keeps all), None (no penalty)
C (Regularization Strength)	Smaller = stronger regularization (simpler model), larger = weaker
Balance Class Distribution	Enabled = better for imbalanced data, Disabled = equal class treatment

Scatter Plot Analysis

Reference: [ChatGPT Analysis of Scatter Plots](#)

Histograms, Boxplots, and Statistical Information

Reference: [ChatGPT Analysis of Distributions](#)

Outlier Visualization (ChatGPT-4o)

Question: When is the best time to use outlier visualization techniques?

Reference: [ChatGPT Discussion on Outlier Detection](#)

Dataset Class Analysis (Perplexity)

Reference: [Are the classes in the dataset balanced?](#)

Image Analysis (Perplexity)

Reference: [Analyze these images for patterns/outliers](#)